

2. Multi-armed bandits



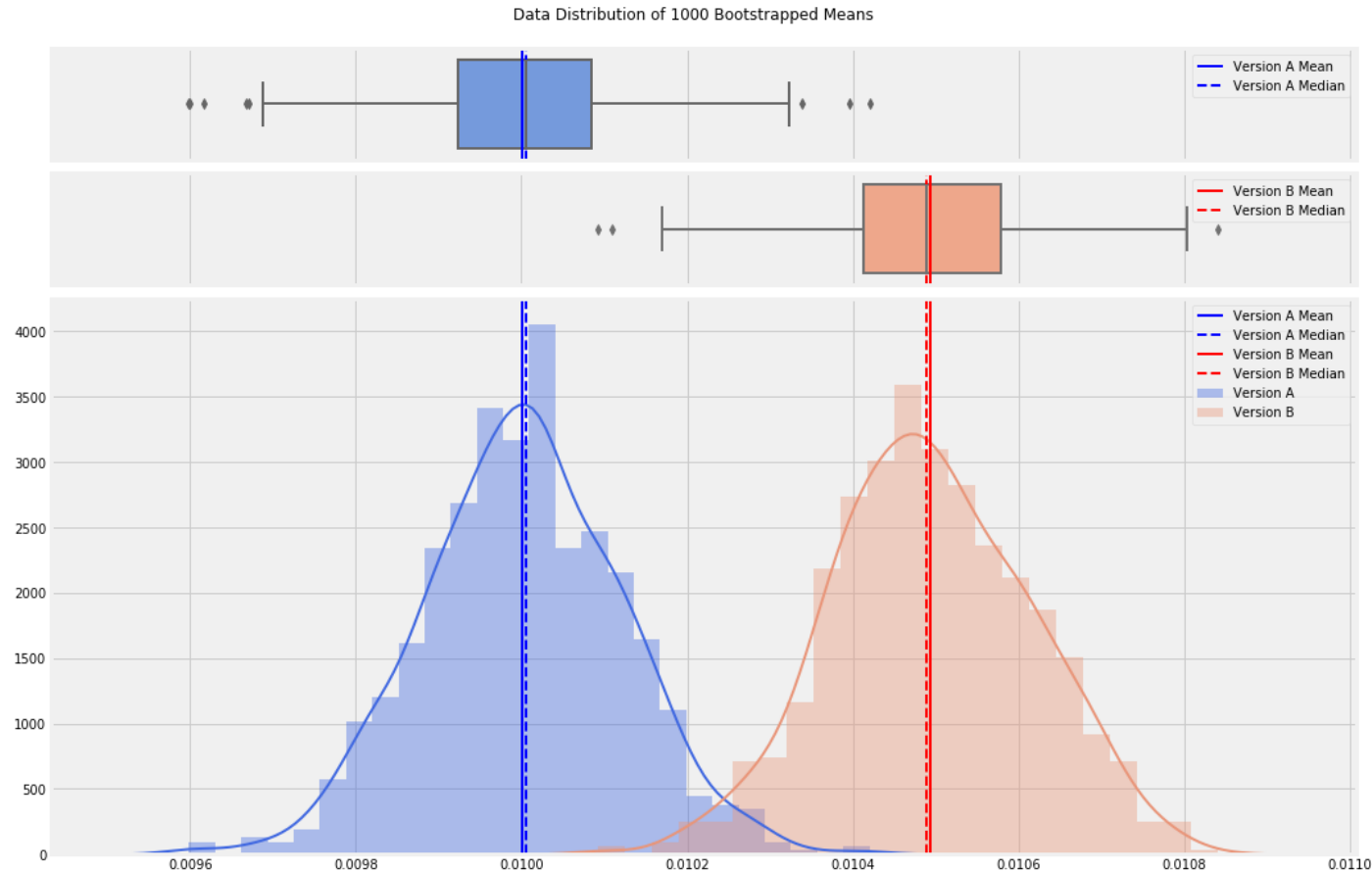
Possible sequential choices of a bee: land on white or yellow flowers.



Rewards are typically random and unknown in advance:

$$f_{r|w}(r|a = \text{white}) \quad f_{r|y}(r|a = \text{yellow})$$

Traditional A/B testing: determine the average value of each distribution by selecting each arm $t/|\mathcal{A}|$ times until some statistical criterion (e.g. p-value) is met.



Drawback: if one choice is vastly inferior to another, yet you must continue to select that choice until the test is complete, thus losing potential gains.

Possible sequential choices of a bee: land on white or yellow flowers.



Online decision making: we want to explore the different options over time to learn what their payouts are, but simultaneously we want to exploit the highest paying option.

Choices are based on previous experience $H_{t-1} = \{a_1, r_1, \dots, a_{t-1}, r_{t-1}\}$, with the aim of maximizing expected rewards (in drops of nectar).

Richard Feynman had lunch in 1970 with his friend Ralph Leighton in a Vietnamese restaurant.

They posed the following question to themselves: *should we stick to the dish we already know and like, or explore some other in order to maximize our pleasure. And how should we do this?*



https://www.feynmanlectures.caltech.edu/info/solutions/restaurant_problem_sol_1.html

<https://economictimes.indiatimes.com/news/international/us/a-nobel-physicists-lunch-order-became-math-and-it-took-50-years-to-crack-the-code/articleshow/131800472.cms?from=mdr>

Online decision-making involves a trade-off between:

- **Exploitation**, take most rewarding action, given the current information,
- **Exploration**, gather more information to take better decisions in the future,

through a number of trials t (**earning vs. learning, control vs. estimation**)

Getting to know the best long-term strategy may involve short-term sacrifices.

Settings

Stochastic environment: rewards are sampled from an unknown product distribution. Rewards are i.i.d.

Adversarial environment: rewards are chosen arbitrarily by an adversary which, at time t , knows all past actions taken, but not the currently selected one and has complete control over the rewards. We cannot make statistical assumptions whatsoever about the nature of the process generating rewards.

Examples

Which ones are stochastic and which ones adversarial?

- Medical treatment

Exploitation Use the known treatment

Exploration Try an experimental one

- On line advertising

Exploitation Show the most succesful advert

Exploration Show a different advert

- Oil drilling

Exploitation Drill at the best known location

Exploration Drill a new location

- Game playing

Exploitation Play the move you believe is best

Exploration Play an experimental move

- Sales planning

Exploitation Use last season strategy

Exploration Try new offers

The stochastic m -armed bandit problem

Let us formalize the concept...

A **stochastic multi-armed bandit** is defined by a tuple $\langle \mathcal{A}, \mathcal{R} \rangle$

- $\mathcal{A}$ is a set of $m = |\mathcal{A}|$ actions (or “arms”)
- At each step t
 - agent selects one action $a_t \in \mathcal{A}$
 - environment generates a reward r_t
- The reward follows an unknown probability density $r \sim f(r|a)$
- Actions do not affect future rewards
- The objective is to take appropriate decisions so as to maximize the cumulative reward



$$R_t = \sum_{\tau=1}^t r_{\tau}$$

- The **action-value** is the mean reward for action a : $Q(a) = E \{ r \mid A = a \}$
- The **optimum value** is: $V^* = Q(a^*) = \max_{a \in \mathcal{A}} Q(a)$
- The **regret** is the opportunity loss for one step: $l_t = E \{ V^* - r_t \}$

- The **total regret** is defined as the total opportunity loss:

$$\begin{aligned}
 L_t &= E \left\{ \sum_{\tau=1}^t (V^* - r_\tau) \right\} = \sum_{a \in \mathcal{A}} E \{ N_t(a) \} (V^* - Q(a)) \\
 &= \sum_{a \in \mathcal{A}} E \{ N_t(a) \} \Delta_a
 \end{aligned}$$

Number of pulls for arm a up to time t

Minimizing the total regret is equivalent to maximizing the cumulative reward.

A good algorithm ensures not too frequent selection of arms with large gaps Δ_a .
But gaps are unknown!

Proof of the expression for the total regret...

$$\begin{aligned}
 L_t &= E_r \left\{ \sum_{\tau=1}^t (V^* - r_\tau) \right\} \\
 &= \sum_{\tau=1}^t E_r \{ V^* - r_\tau \} \\
 &= \sum_{\tau=1}^t \sum_{a \in \mathcal{A}} E_{r|a} \{ (V^* - r_\tau) | A_\tau = a \} p(A_\tau = a) \\
 &= \sum_{\tau=1}^t \sum_{a \in \mathcal{A}} V^* 1_{A_\tau=a} p(A_\tau = a) - \sum_{\tau=1}^t \sum_{a \in \mathcal{A}} Q(a) 1_{A_\tau=a} p(A_\tau = a) \\
 &= \sum_{\tau=1}^t \sum_{a \in \mathcal{A}} 1_{A_\tau=a} p(A_\tau = a) (V^* - Q(a)) \\
 &= \sum_{a \in \mathcal{A}} E \{ N_t(a) \} (V^* - Q(a)) \\
 &= \sum_{a \in \mathcal{A}} E \{ N_t(a) \} \Delta_a
 \end{aligned}$$

Probability of pulling arm a at time τ

Indicator function: 1 if the argument is true, 0 otherwise

Minimizing the total regret is equivalent to maximizing the cumulative reward.

Number of pulls for arm a up to time t

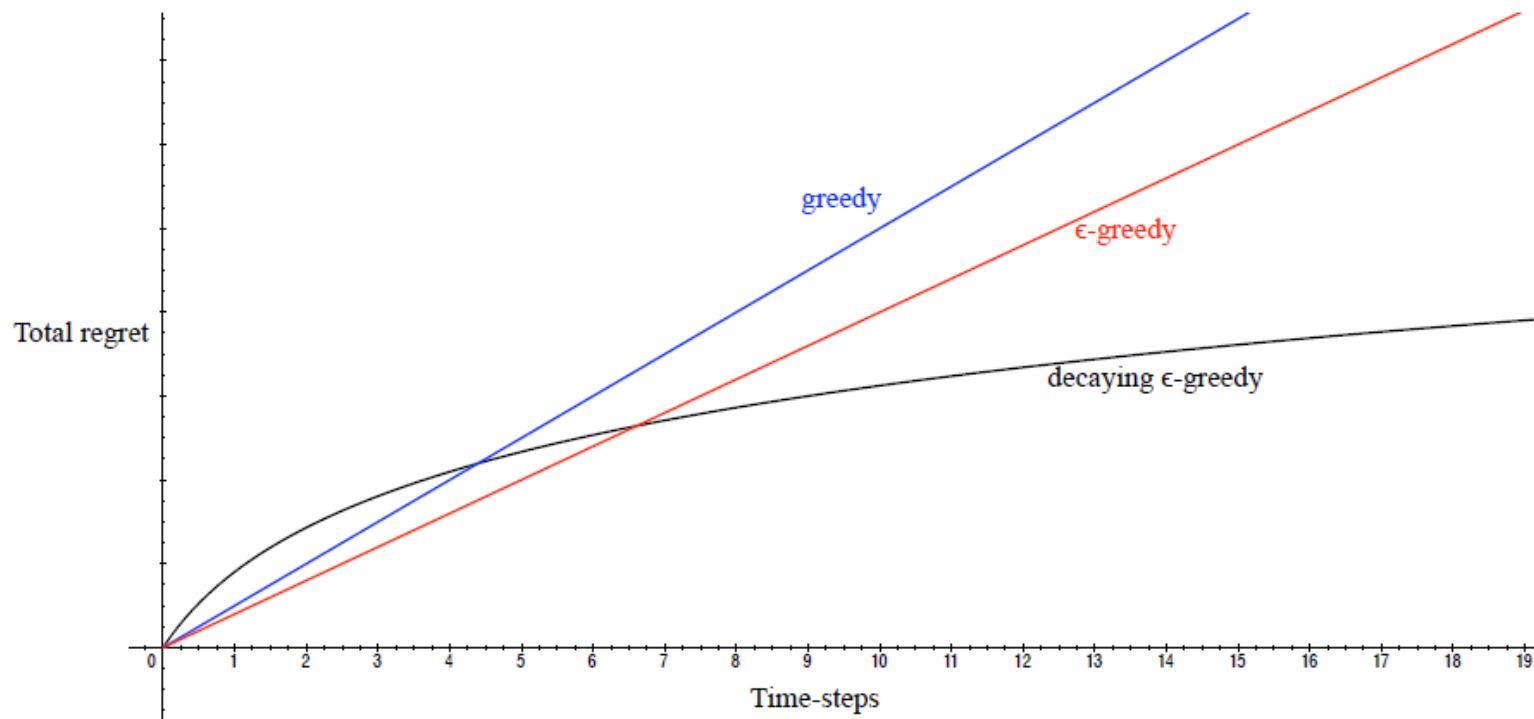
What is the total regret for the traditional A-B testing? $L_t = \frac{t}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} (V^* - Q(a))$

Linear growth in time

How to compute the total regret in practice...

$$L_t = E \left\{ \sum_{\tau=1}^t (V^* - r_{\tau}) \right\} \approx \frac{1}{N_{runs}} \sum_{i=1}^{N_{runs}} \underbrace{L_{t-1}^{(i)} + (V^* - r_t^{(i)})}_{\text{Cumulated regret in a random run}}$$

Cumulated regret in a random run



From David Silver, Course on RL, 2015

Is it possible to achieve sub-linear total regret? Yes!

The problem is that we **do not know the value of each action**. We need to guess it from experience.

At any time step, there are actions that are better than the rest: the *greedy* actions. If we adopt them, we **exploit**. If we choose other, we **explore**.

Both cannot be simultaneous: exploring is worse in terms of reward in the short term, but pays off in the long term.

How to balance between them? Four approaches:

1. Random exploration
2. Upper-confidence-bound action selection
3. Soft-max strategy and gradient bandits
4. Probability matching

Action-value estimation

How to estimate values of actions? By averaging the observed rewards:

$$\hat{Q}_t(a) = \frac{1}{N_t(a)} \sum_{\tau=1}^t r_{\tau}^a$$

Number of pulls for arm a up to time t

Greedy action selection: $a_t = \arg \max_a \hat{Q}_t(a)$

The agent can get stuck in a non-optimal action for ever. Then, the regret grows linearly in time.

Approach 1. Random exploration

Random exploration technique:

ε -greedy action selection:
$$a_t = \begin{cases} \arg \max_a \hat{Q}_t(a) & \text{with probability } 1 - \varepsilon \\ \text{random action} & \text{with probability } \varepsilon \end{cases}$$

We will be making errors and regret will still be linear (every possible action will be sampled an infinite number of times as $t \rightarrow \infty$), unless...

...taking a decaying schedule for ε :

$$\varepsilon_t = \min \left\{ 1, \frac{\delta |\mathcal{A}|}{t} \right\} \quad \delta > 0$$

we obtain a logarithmic asymptotic total regret provided that (for rewards bounded to $[0,1]$):

$$\Delta_a = V^* - Q(a) \quad \delta \geq \frac{1}{\min_{a \neq a^*} \Delta_a^2}$$

Best performance if δ is close to this bound [Auer 2002]

Simple ε -greedy bandit algorithm

Initialize

for $i = 1:m$

$Q(i) = \text{rand}$

$N(i) = 0$

$t = 0$

Loop forever

$t \leftarrow t + 1$

Update ε

$A \leftarrow \begin{cases} \arg \max_a Q(a) & \text{with probability } 1 - \varepsilon \\ \text{random action} & \text{with probability } \varepsilon \end{cases}$

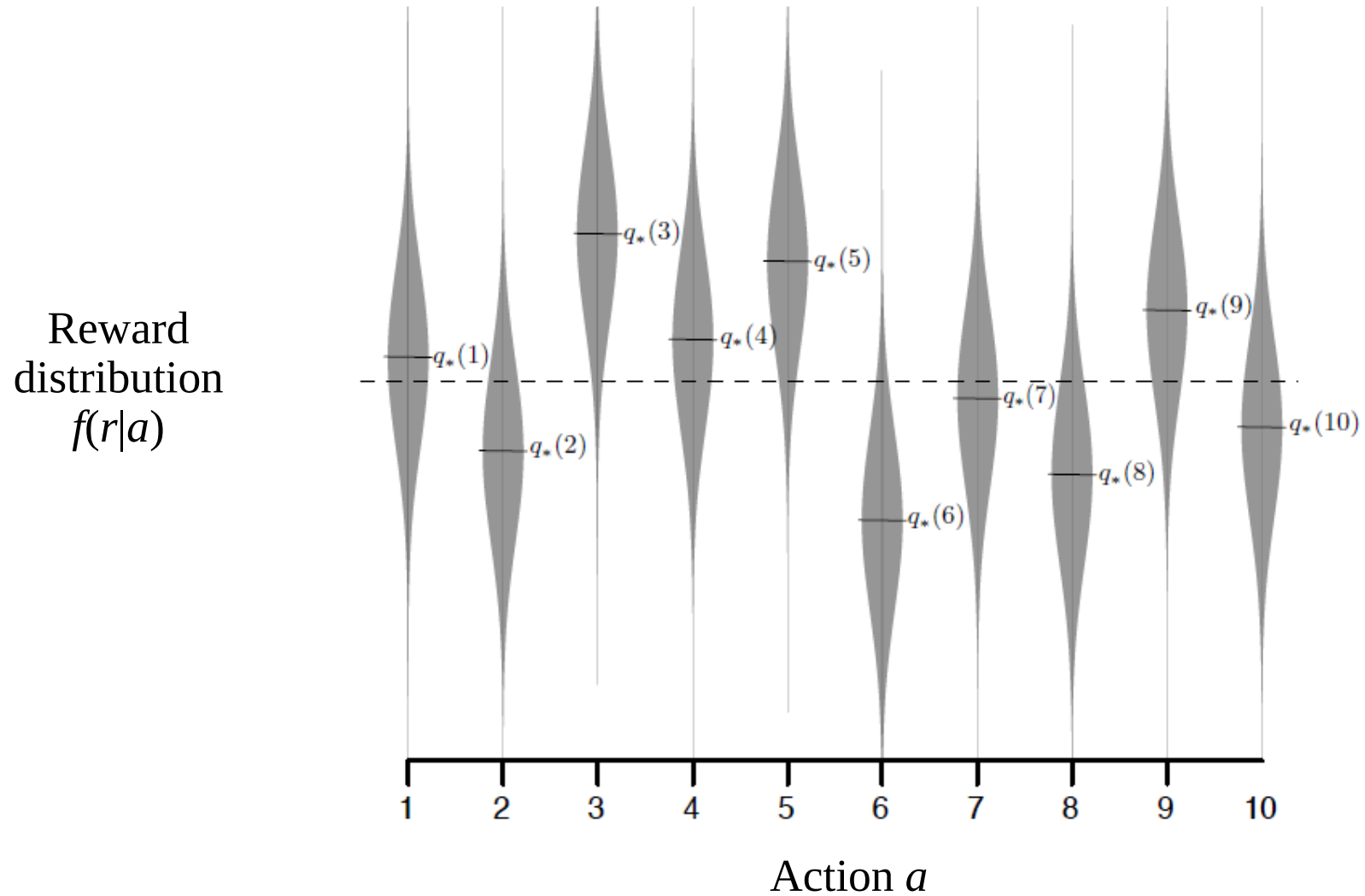
$r \leftarrow \text{bandit}(A)$

$N(A) \leftarrow N(A) + 1$

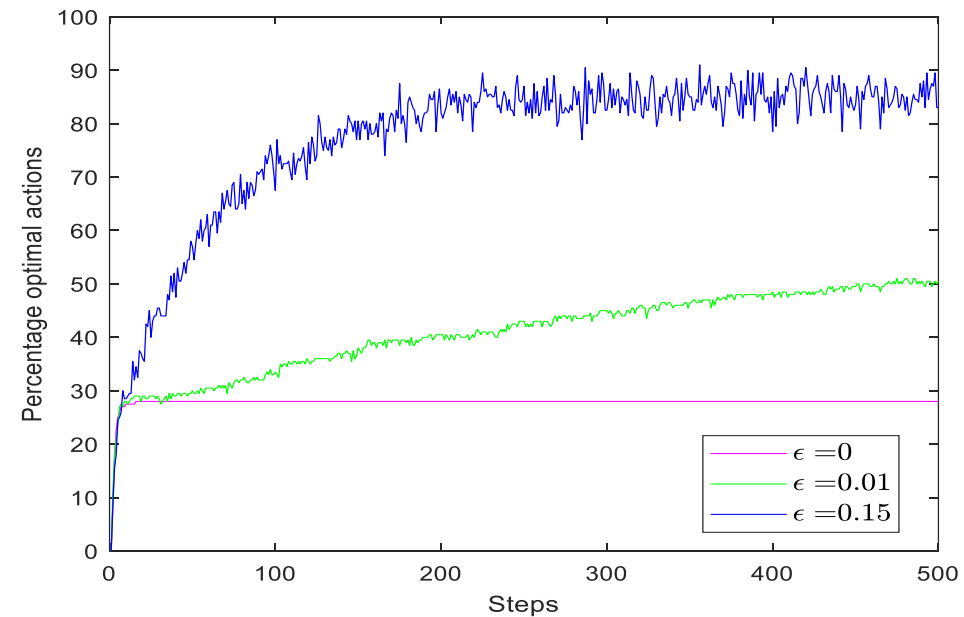
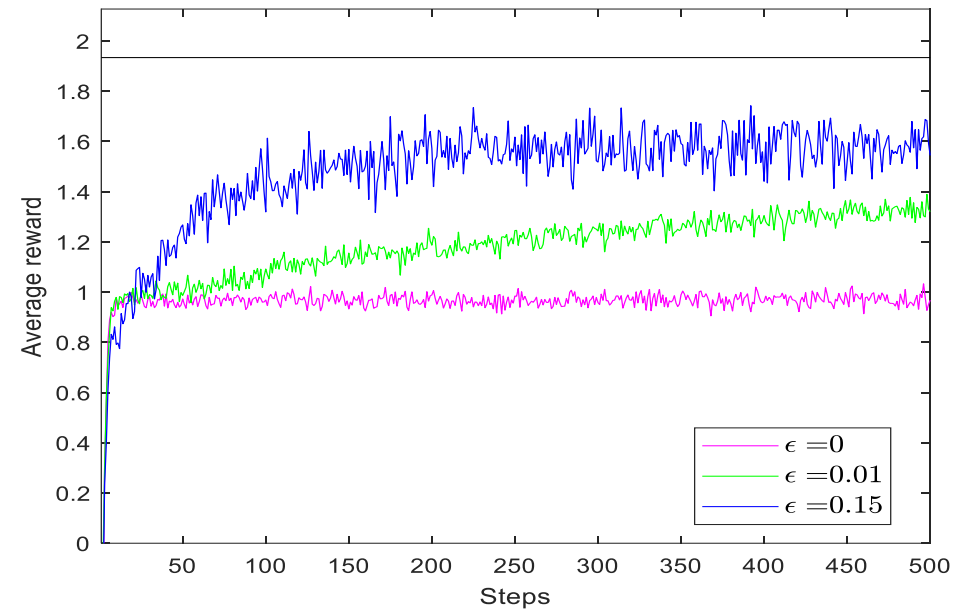
$Q(A) \leftarrow Q(A) + \frac{1}{N(A)}(r - Q(A))$

$$\varepsilon_t = \min \left\{ 1, \frac{\delta |\mathcal{A}|}{t} \right\} \quad \delta > 0$$

Example 2.1. A 10-armed bandit with unit-variance Gaussian rewards



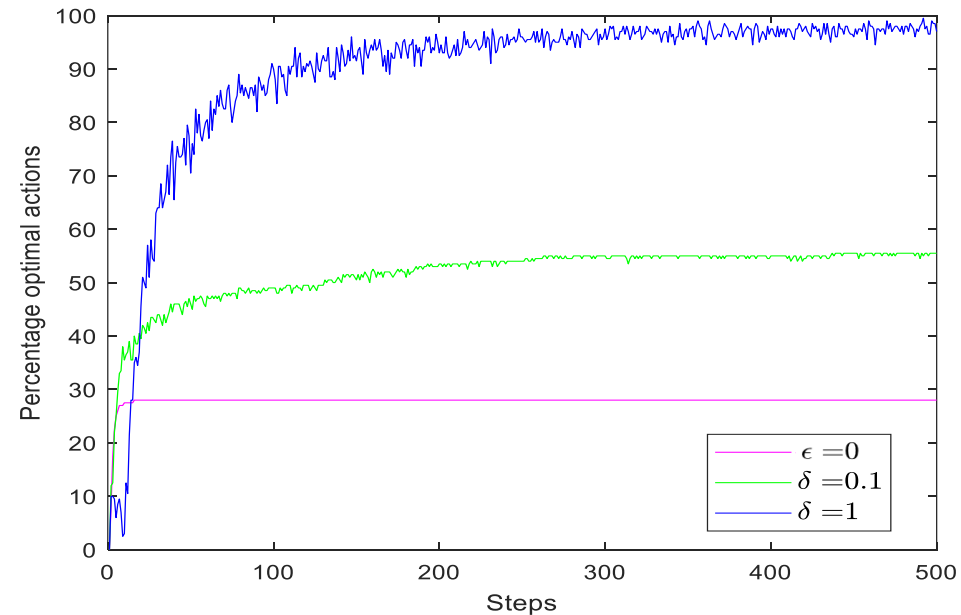
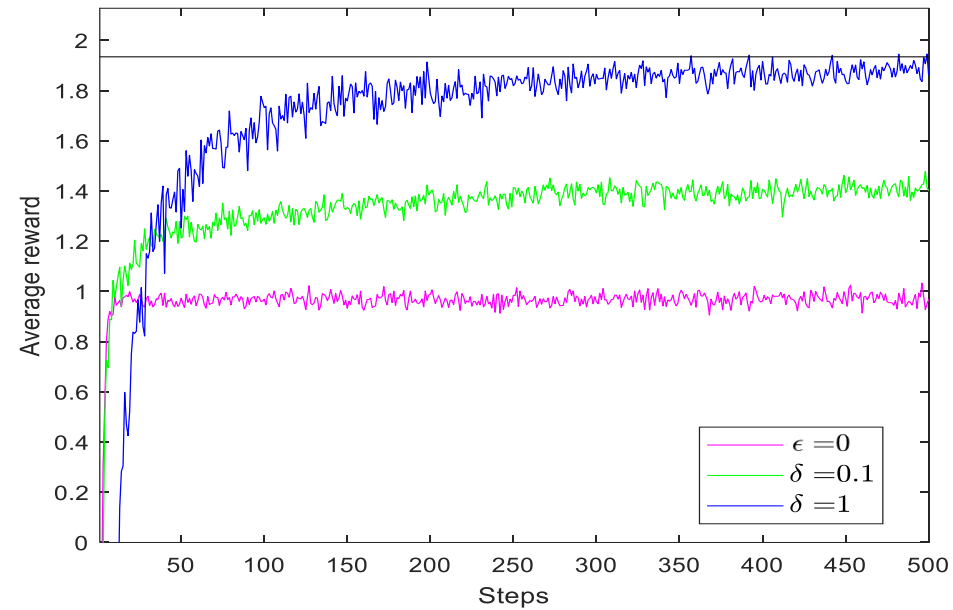
Example 2.1. Performance averaged over 200 runs with **constant** ϵ



Example 2.1. Performance averaged over 200 runs with **decaying ϵ**

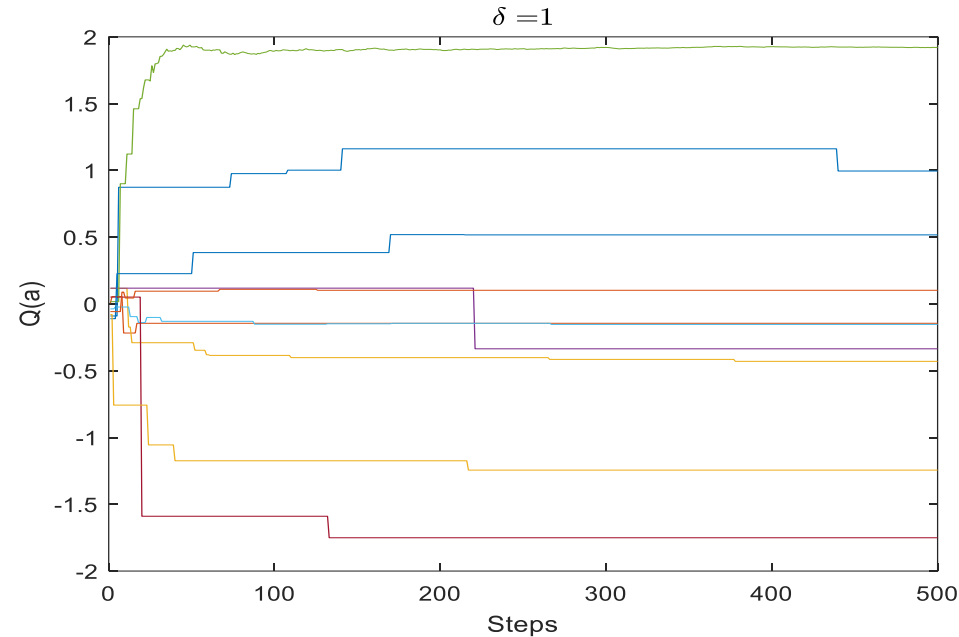
What would the performance be if variances were higher?

What if they were lower?



Example 2.1. Performance averaged over 200 runs with **decaying** ε

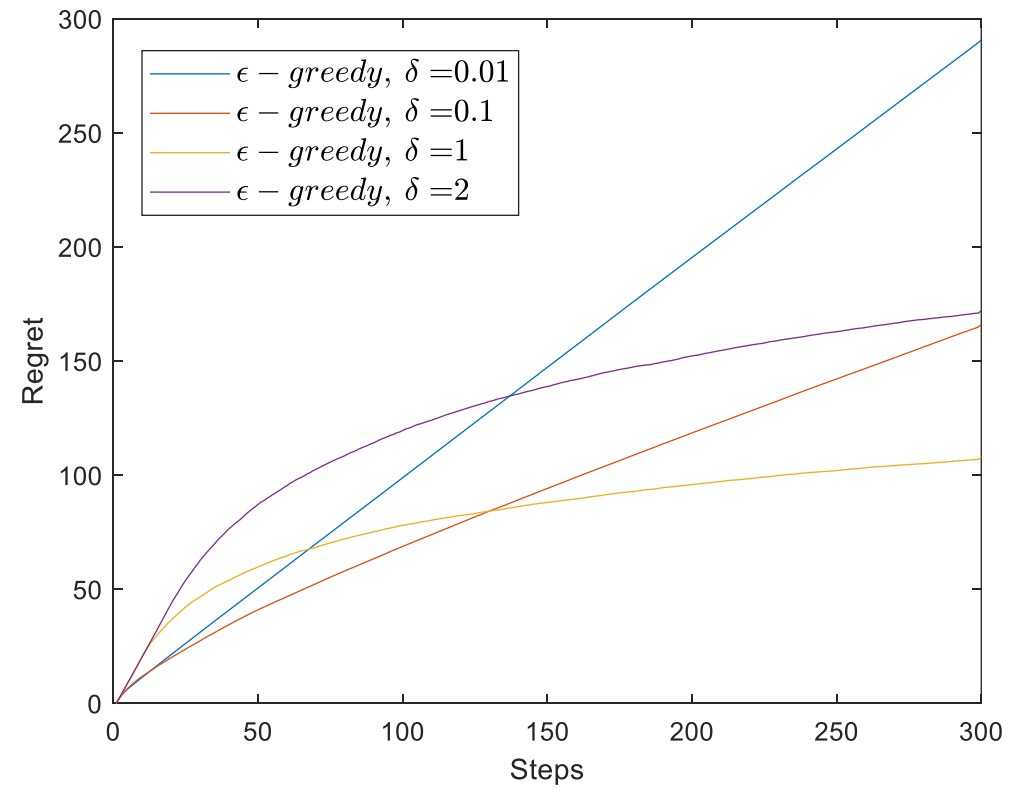
Evolution of the estimated $Q(a)$ on a single run



True means $E\{r|a\}$: 0.8782 -0.1503 -0.4655 -0.8348 1.9340
 -0.2624 -2.5056 0.7514 0.1667 -1.4844

Check the bound on δ

Example 2.1. ϵ - greedy regret



Lower performance bound

The performance of any decision algorithm depends on the similarity between the pdf of optimal arm $f(r|a^*)$ and other arms: hard problems have similarly looking arms with different means.

Theorem (Lai and Robbins)

Asymptotical total lower bound:

$$\lim_{t \rightarrow \infty} L_t \geq \log t \sum_{a \neq a^*} \frac{\Delta_a}{D(f(r|a) \| f(r|a^*))} \approx O\left(\sum_{a \neq a^*} \frac{1}{\Delta_a}\right)$$

where D is the Kullback-Leibler divergence between two density functions, and

$$\Delta_a = V^* - Q(a)$$

An algorithm is said asymptotically optimal if it achieves the bound (UCB and probability matching are).

Incremental implementation and tracking

Some exploration level is needed if average rewards change over time: $\varepsilon > 0$ and online recomputing of $Q(a)$.

An implementation of estimated $Q(a)$ that requires constant memory and computation effort is:

$$\begin{aligned}\hat{Q}_{t+1}(a) &= \frac{1}{N_t(a)} \sum_{i=1}^{N_t(a)} r_i = \frac{1}{N_t(a)} \left(r_t + \sum_{i=1}^{N_t(a)-1} r_i \right) \\ &= \frac{1}{N_t(a)} \left(r_t + (N_t(a)-1) \frac{1}{(N_t(a)-1)} \sum_{i=1}^{N_t(a)-1} r_i \right) \\ &= \frac{1}{N_t(a)} \left(r_t + (N_t(a)-1) \hat{Q}_t(a) \right) = \frac{1}{N_t(a)} \left(r_t + N_t(a) \hat{Q}_t(a) - \hat{Q}_t(a) \right) \\ &= \hat{Q}_t(a) + \frac{1}{N_t(a)} \underbrace{\left(r_t - \hat{Q}_t(a) \right)}_{\text{Error in the estimate}}\end{aligned}$$

Number of times action a has been selected up to time t

where r_i are the rewards obtained whenever a is the selected action.

If **rewards change over time** we can exploit this implementation giving more weight to recent rewards than to long-past rewards:

$$\hat{Q}_{t+1}(a) = \hat{Q}_t(a) + \alpha (r_t - \hat{Q}_t(a))$$

where the step size $\alpha \in (0,1]$ may be different for every action.

We can even change the value of α over the iterations. To ensure convergence to $Q(a)$ with probability 1:

$$\sum_{t=1}^{\infty} \alpha_t = \infty \qquad \sum_{t=1}^{\infty} \alpha_t^2 < \infty$$

These conditions are met for $\alpha_t = 1/t$.

For a constant step size the condition is not met indicating that the estimates never converge completely but continue to vary in response to the recently received rewards.

For a constant value of α , how does the estimate depend on the initial value?

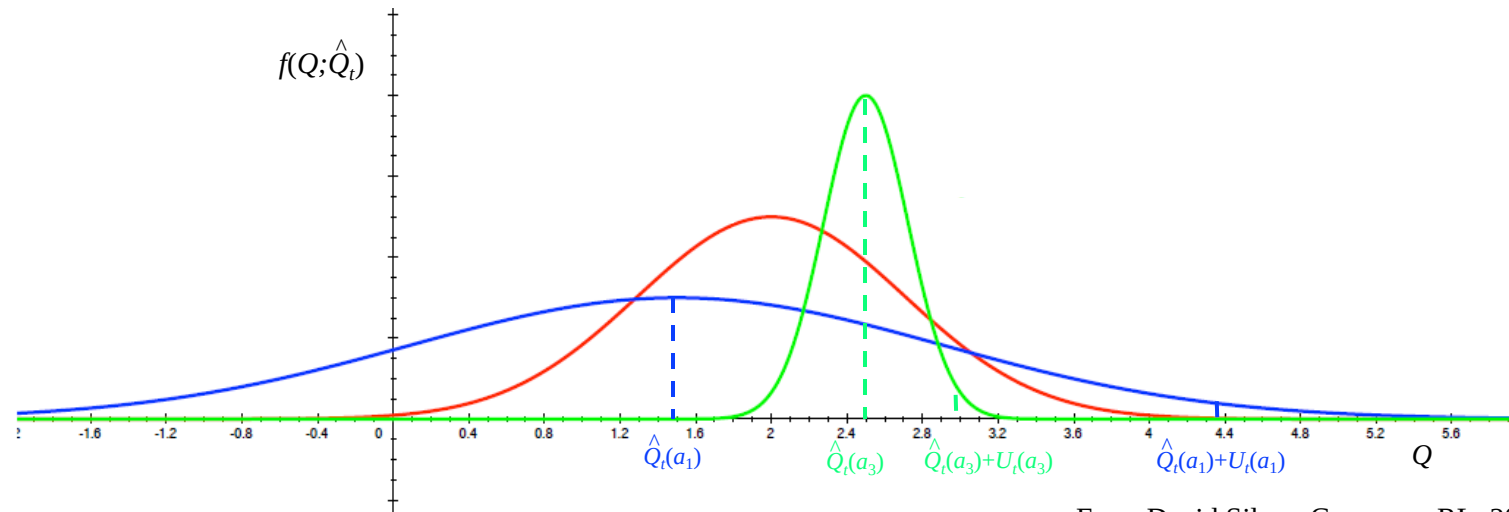
$$\begin{aligned}\hat{Q}_{t+1}(a) &= \hat{Q}_t(a) + \alpha(r_t - \hat{Q}_t(a)) \\ &= \alpha r_t + (1 - \alpha) \hat{Q}_t(a) \\ &= \alpha r_t + (1 - \alpha) (\alpha r_{t-1} + (1 - \alpha) \hat{Q}_{t-1}(a)) \\ &= \alpha r_t + (1 - \alpha) \alpha r_{t-1} + (1 - \alpha)^2 \hat{Q}_{t-1}(a) \\ &= \alpha r_t + (1 - \alpha) \alpha r_{t-1} + (1 - \alpha)^2 \alpha r_{t-2} + \dots + (1 - \alpha)^{t-1} \alpha r_1 + (1 - \alpha)^t \hat{Q}_1(a) \\ &= (1 - \alpha)^t \hat{Q}_1(a) + \sum_{\tau=1}^t \alpha (1 - \alpha)^{t-\tau} r_\tau\end{aligned}$$

The initial value contributes with a bias that fades to zero over time. Check that $\lim_{t \rightarrow \infty} E \{ \hat{Q}_{t+1}(a) \} = E \{ r \mid a \}$

Utterly large **optimistic initial values** is a good strategy for exploration when $\varepsilon = 0$. **Why?**

Approach 2. Upper-confidence-bound action selection

The ϵ -greedy strategy treats all possible non-greedy actions equally during exploration. It would be wiser to select those actions having more potential for optimality.

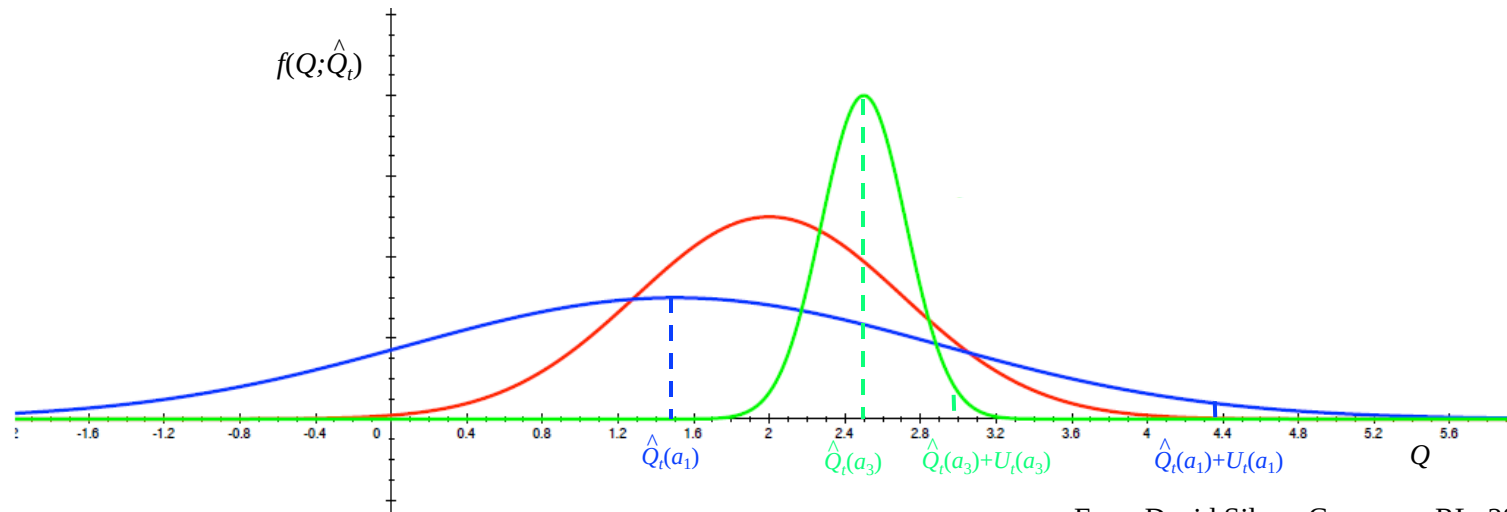


From David Silver, Course on RL, 2015

Which action should we pick?

Approach 2. Upper-confidence-bound action selection

Note that a_1 has the potential of being more rewarding than a_3 ...



From David Silver, Course on RL, 2015

The more uncertain we are about an action-value the more important is to explore it. It could be the best action! **Optimism in the face of uncertainty.**

Approach 2. Upper-confidence-bound action selection

Let us change the rule for picking an action:

- Define an upper confidence bound $U_t(a)$ for each action value, such that:

$$\Pr \left\{ Q(a) \geq \hat{Q}_t(a) + U_t(a) \right\} = \gamma$$

or alternatively, if the density $f(\hat{Q}_t(a), Q(a))$ is symmetric around the mean:

$$\Pr \left\{ Q(a) < \hat{Q}_t(a) - U_t(a) \right\} = \gamma$$

This γ depends on the number of times that a has been selected so far, $N(a)$ (see Annex 1 for a derivation of $U_t(a)$).

- Select actions maximizing the Upper Confidence Bound (UCB) algorithm:

$$a_t \doteq \arg \max_a \left[\hat{Q}_t(a) + U_t(a) \right]$$

which naturally balances **exploration and exploitation**. An action is selected if either has a high estimated mean or if there is much uncertainty so $U(a)$ is large.

Approach 2. Upper-confidence-bound action selection

For finite-domain random variables:

$$a_t \doteq \arg \max_a \left[\hat{Q}_t(a) + \underbrace{\sqrt{\frac{c \left(r_a^M - r_a^m \right)^2 \ln t}{2N_t(a)}}}_{\text{Related to the uncertainty in the estimate of } Q(a)} \right]$$

$c > 0$ controls the level of exploration

Maximum and minimum values of the reward of a random variable with limited domain

Note that uncertainty decreases with $N_t(a)$, the number of times that action a has been selected prior to time t .

UCB achieves logarithmic regret uniformly over time (find the proof [here](#)):

$$L_t \leq \sum_{a \neq a^*} \frac{4c \ln t}{\Delta_a} + \frac{2c}{c-1} \Delta_a$$

Exploration mostly occurs at the beginning of the learning process.

Approach 2. Upper-confidence-bound action selection

For a Gaussian random variable:

$c > 0$ controls the level of exploration

Variance of reward for action a , can be estimated from the observations of r

$$a_t \doteq \arg \max_a \left[\hat{Q}_t(a) + \underbrace{\sqrt{\frac{c \sigma_a^2 \ln t}{N_t(a)}}}_{\text{Related to the uncertainty in the estimate of } Q(a)} \right]$$

Related to the uncertainty in the estimate of $Q(a)$

How to make it work in a non-stationary scenario?

Consider modifying the way we estimate $Q(a)$, the values of $N_t(a)$ and $\ln t$ (you may find bounds in <https://arxiv.org/pdf/0805.3415.pdf>)

UCB algorithm

Initialize

for $i = 1:m$

$Q(i) = \text{rand}$

$N(i) = 0.0001$ % Small value to avoid divides by zero

$t = 0$

Loop forever

$t \leftarrow t + 1$

$$A \leftarrow \arg \max_a \left[Q(a) + \sqrt{\frac{c \left(r_a^M - r_a^m \right)^2 \ln t}{2N(a)}} \right]$$

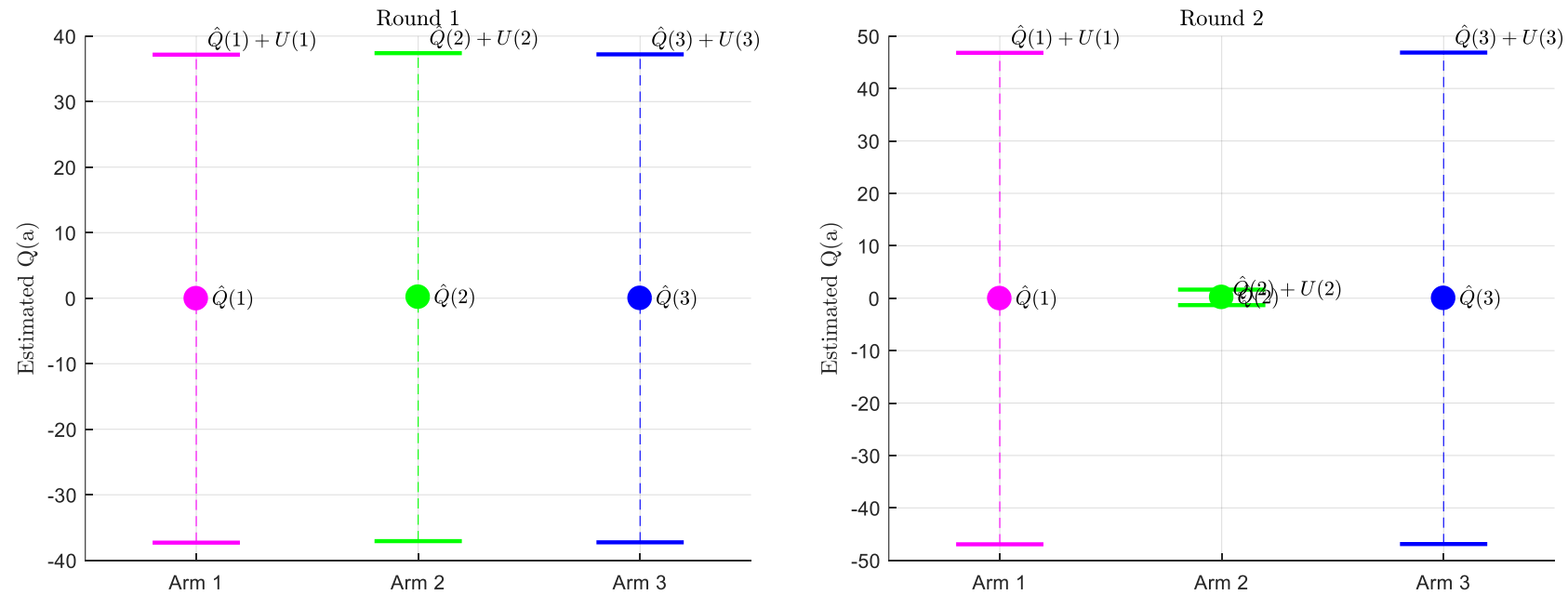
$r \leftarrow \text{bandit}(A)$

$N(A) \leftarrow N(A) + 1$

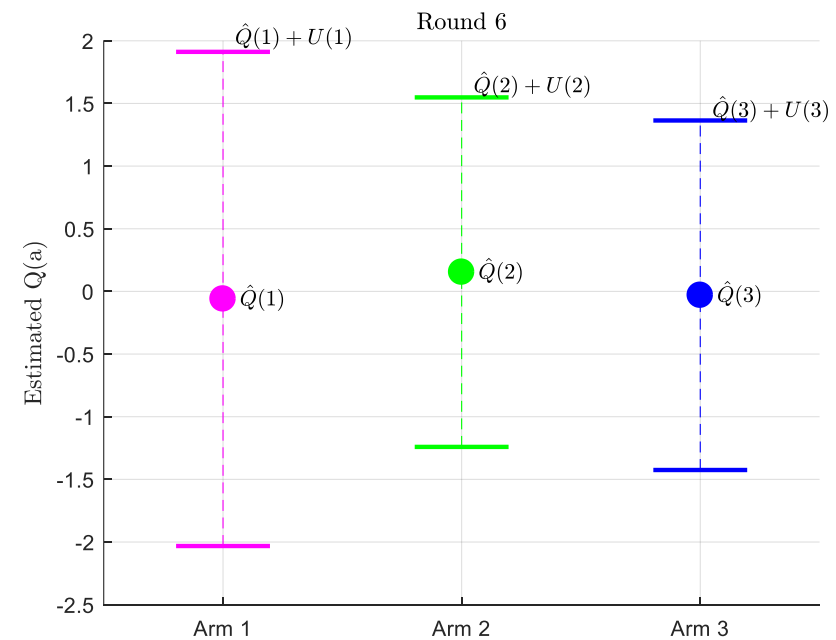
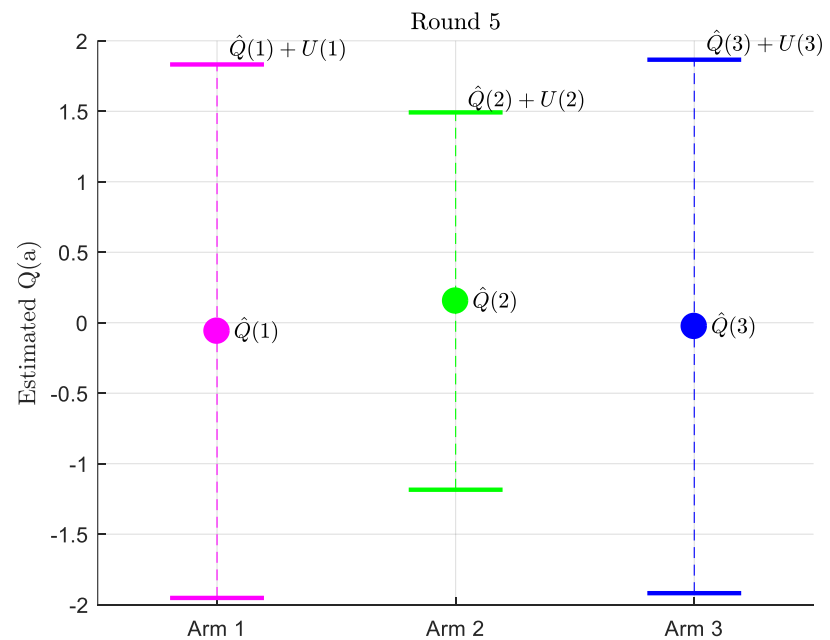
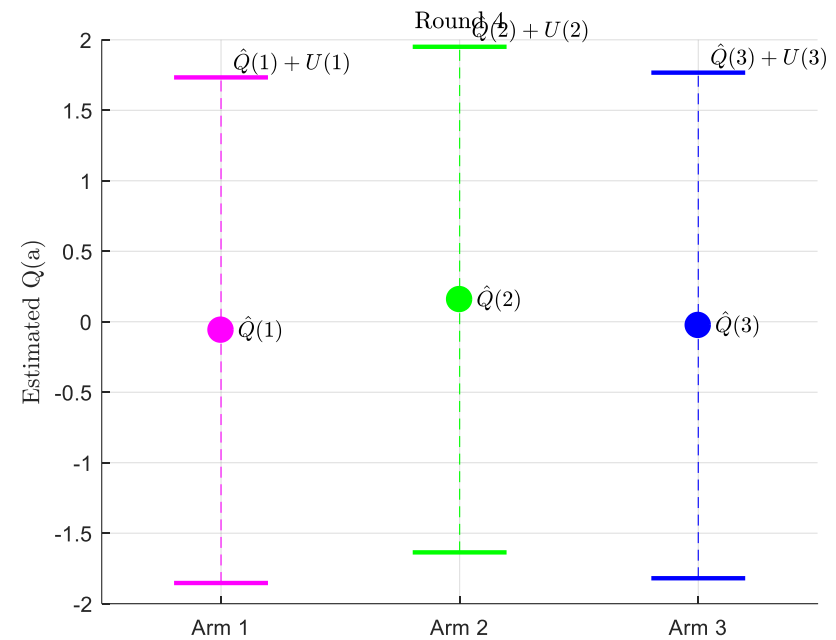
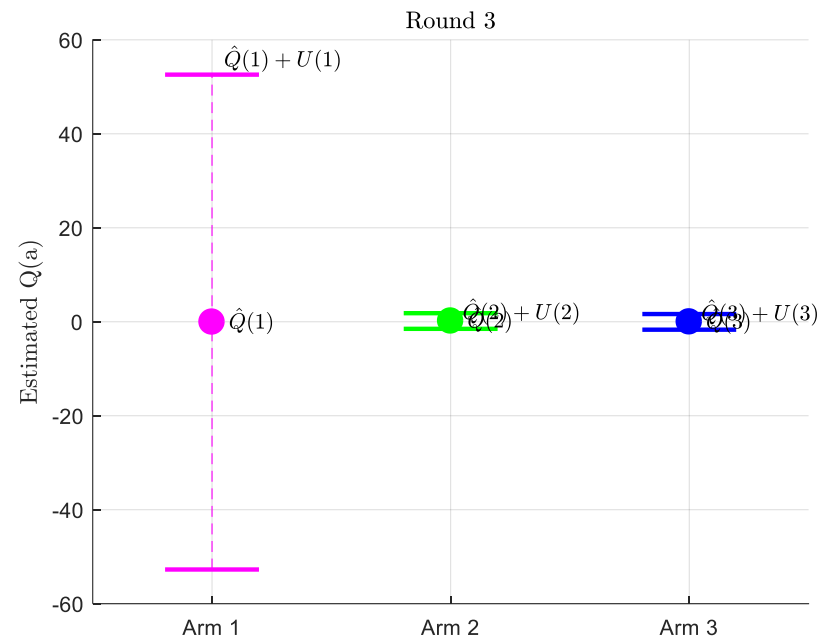
$$Q(A) \leftarrow Q(A) + \frac{1}{N(A)} (r - Q(A))$$

Why does it work?

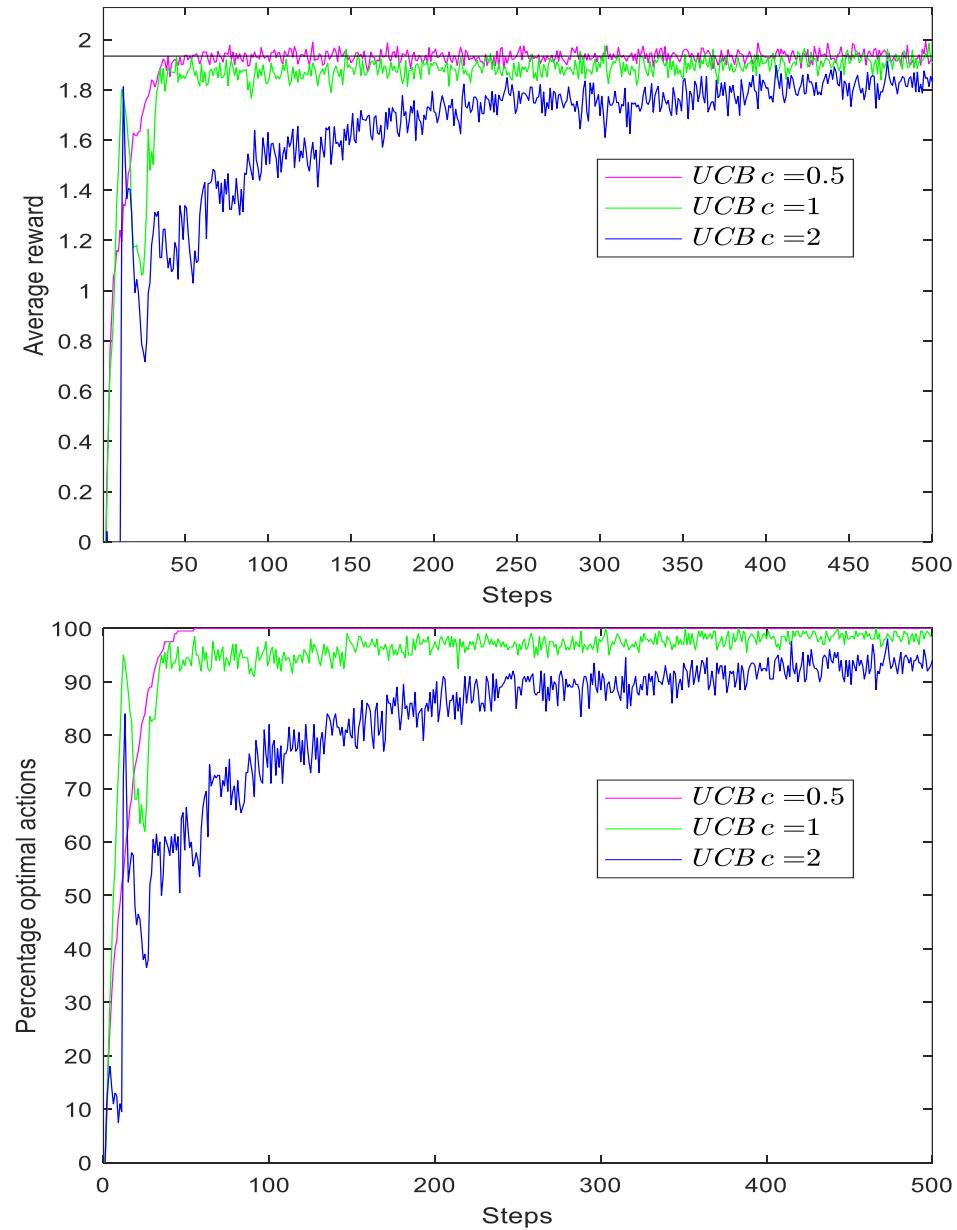
The evolution of the estimated $Q(a)$ and the uncertainty in a 3 arms case with Gaussian rewards.



Guess which arm is selected upon each time step.

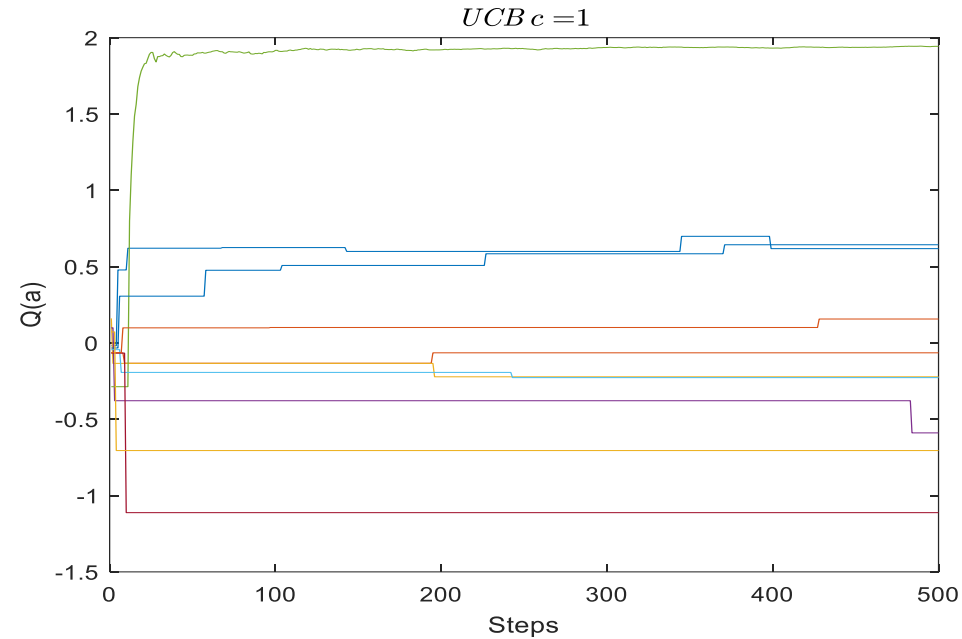


Example 2.1. UCB convergence averaged over 200 runs



Example 2.1. UCB convergence

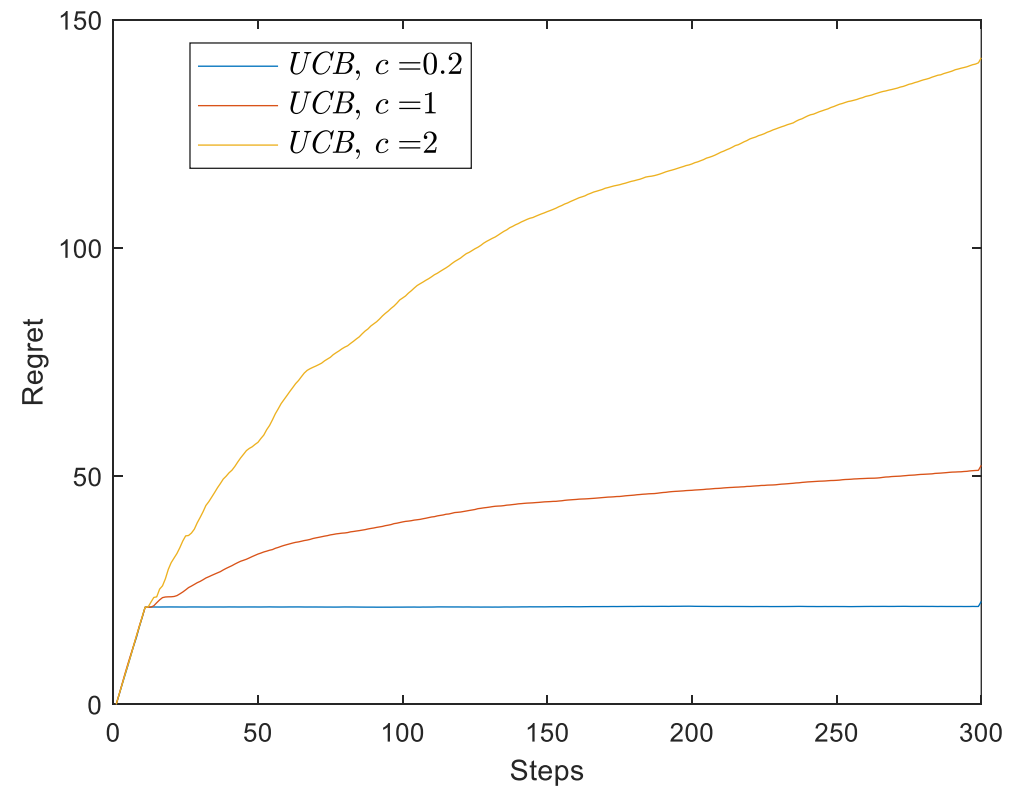
Evolution of the estimated $Q(a)$ on a single run



True $Q(a) = E\{r|a\}$: 0.8782 -0.1503 -0.4655 -0.8348 1.9340
 -0.2624 -2.5056 0.7514 0.1667 -1.4844

Why only the largest one is correctly estimated?

Example 2.1. UCB regret



Approach 3. Soft-max strategy

An alternative way of addressing the drawbacks of ε -greedy approach is to use a soft-max strategy adopting a [Boltzmann distribution](#), where actions are selected according to the following probability distribution:

$$\pi_t(a) = \frac{\exp(\hat{Q}_t(a)/\gamma)}{\sum_{y \in \mathcal{A}} \exp(\hat{Q}_t(y)/\gamma)}$$

The value of $\gamma > 0$ denotes the *temperature* of the system:

- $\gamma \rightarrow 0$ represents no exploration at all
- $\gamma \rightarrow \infty$ reflects a choice of action values with almost equal probability

We can think of an adaptive strategy that reduces γ over time.

Beware when computing exponentials!

SoftMax algorithm

Initialize

for $i = 1:m$

$Q(i) = \text{rand}$

$N(i) = 0$

$t = 0$

Loop forever

$t \leftarrow t + 1$

$$\pi(a) = \frac{e^{Q(a)/\gamma}}{\sum_{y \in \mathcal{A}} e^{Q(y)/\gamma}}$$

$A \sim \pi(a)$

$r \leftarrow \text{bandit}(A)$

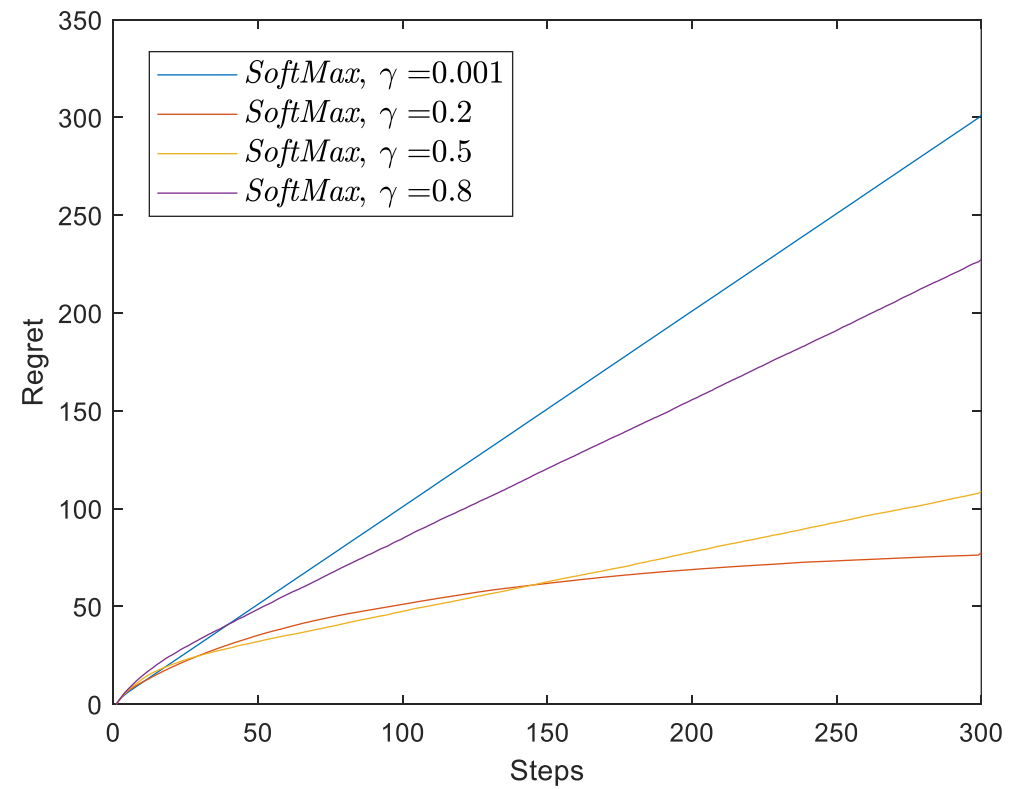
$N(A) \leftarrow N(A) + 1$

$$Q(A) \leftarrow Q(A) + \frac{1}{N(A)} (r - Q(A))$$

Draw a random action
according to the
distribution $\pi(a)$

Compute $e^{(Q(a) - Q_{\max})/\gamma}$ to avoid overflow

Example 2.1. Softmax regret



... moving towards gradient bandits

Let us define $H_t(a)$ a preference for action a and introduce it in the expression above:

$$\pi_t(a) = \frac{\exp(H_t(a))}{\sum_{y \in \mathcal{A}} \exp(H_t(y))}$$

At each time step, action A_t is randomly selected according to $\pi_t(a)$ and a reward r_t is received. Then, if maximizing $E\{r_t\}$ using a gradient approach, the action preferences are updated as:

$$H_{t+1}(A_t) = H_t(A_t) + \alpha(r_t - \bar{R}_t)(1 - \pi_t(A_t))$$

$$H_{t+1}(a) = H_t(a) - \alpha(r_t - \bar{R}_t)\pi_t(a) \quad \text{for all } a \neq A_t$$

$\bar{R}_t$ is a baseline value

Interpretation: if the reward is higher than the baseline $\bar{R}_t$, the probability of taking A_t in the future is increased, and viceversa. The non-selected actions move in the opposite direction.

See Annex 2 for a proof.

Gradient Bandit algorithm

Initialize

for $a = 1:m$

$Q(a) = \text{rand}$

$H_0(a) = \text{rand}$

$N(a) = 0$

$t = 0$

Loop forever

$t \leftarrow t + 1$

$$\pi(a) = \frac{e^{H(a)}}{\sum_{y \in \mathcal{A}} e^{H(y)}}$$

$A \sim \pi(a)$

$r \leftarrow \text{bandit}(A)$

$N(A) \leftarrow N(A) + 1$

$$Q(A) \leftarrow Q(A) + \frac{1}{N(A)} (r - Q(A))$$

Update $H(a) \quad \forall a \in \mathcal{A}$

Draw a random action
according to the
distribution $\pi(a)$

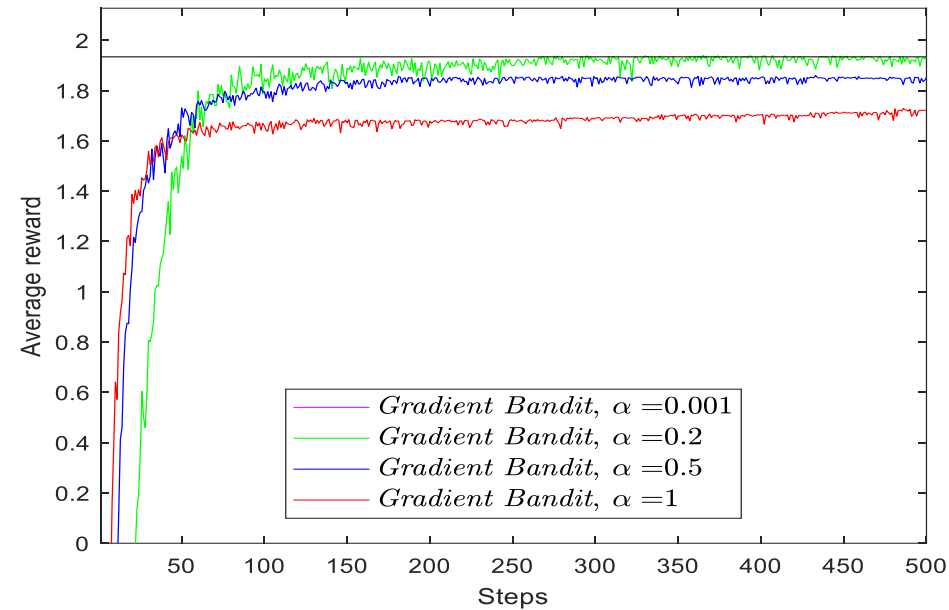
Compute $e^{(H(a) - H_{\max})}$ to avoid overflow

$$\bar{R} \leftarrow \frac{1}{|\mathcal{A}|} \sum_y Q(y)$$

$$H(A) \leftarrow H(A) + \alpha (r - \bar{R}) (1 - \pi(A))$$

$$H(a) \leftarrow H(a) - \alpha (r - \bar{R}) \pi(a) \quad \text{for all } a \neq A$$

Example 2.1. Gradient bandit convergence averaged over 200 runs

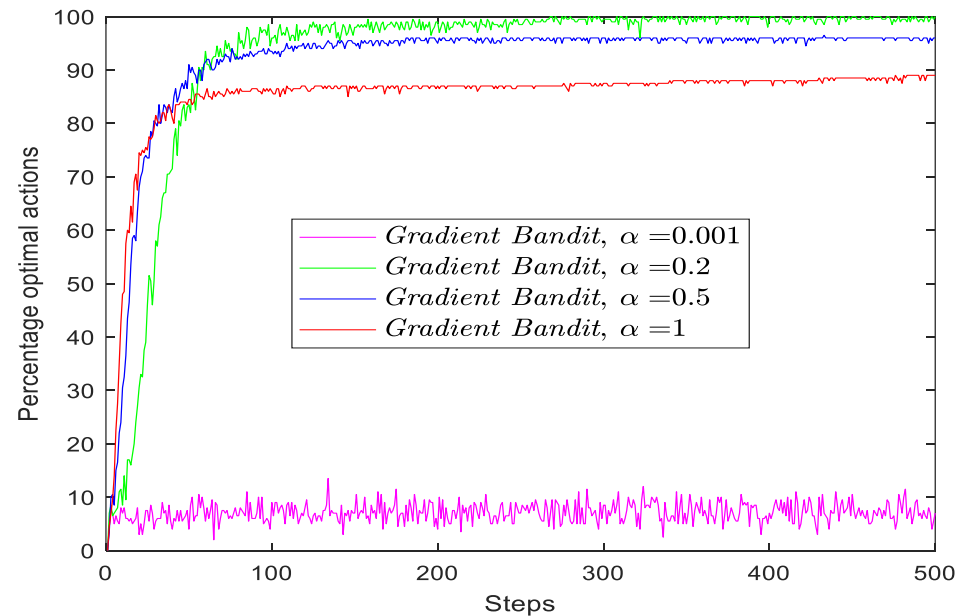


$$\bar{R}_t = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \hat{Q}(a)$$

Convergence is faster if we take

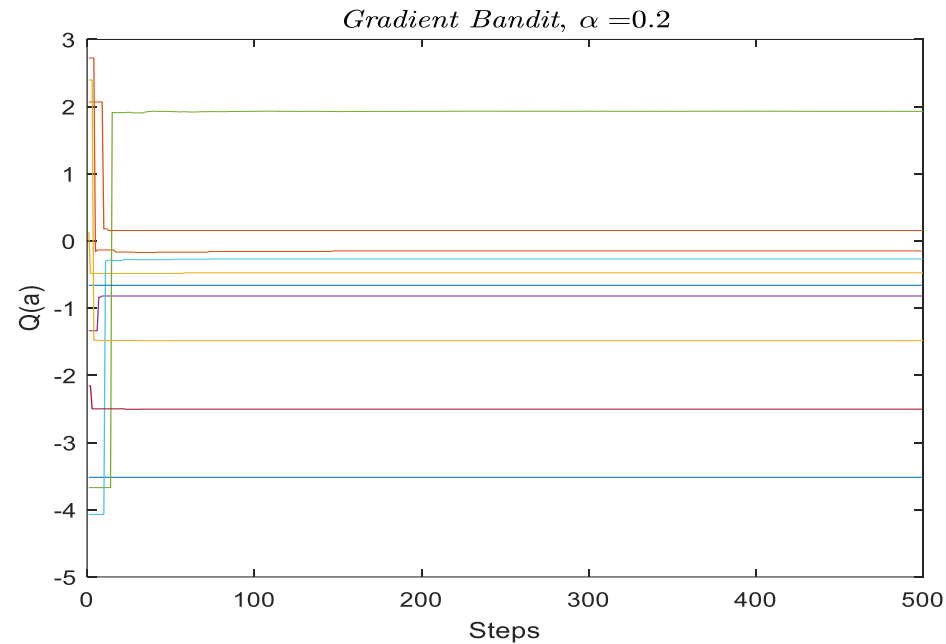
$$\bar{R}_t = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} E\{r | a\}$$

but requires knowing the true mean values



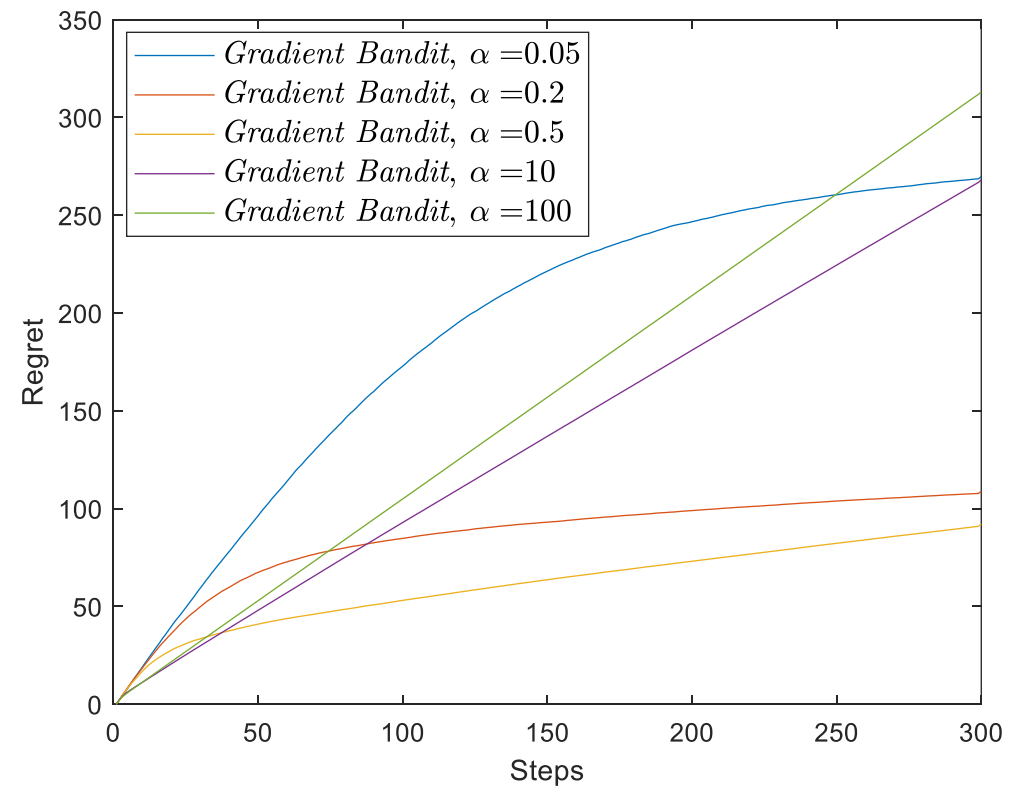
Example 2.1. Gradient bandit convergence

Evolution of the estimated $Q(a)$ on a single run



True means $E\{r|a\}$: 0.8782 -0.1503 -0.4655 -0.8348 1.9340
-0.2624 -2.5056 0.7514 0.1667 -1.4844

Example 2.1. Gradient bandit regret



Approach 4. Probability matching

If we had some prior statistical knowledge about $Q(a)$, how can we introduce it?
Assume we have observed a list of rewards associated to action a :

$$r_{t-1}^a, r_{t-2}^a, \dots, r_1^a$$

The posterior of the average reward $Q(a)$ can be written using Bayes rule:

$$f\left(Q(a) \mid r_t^a, r_{t-1}^a, r_{t-2}^a, \dots, r_1^a\right) \propto f_{r|a}\left(r_t^a \mid Q(a), r_{t-1}^a, r_{t-2}^a, \dots, r_1^a\right) f_Q\left(Q(a) \mid r_{t-1}^a, r_{t-2}^a, \dots, r_1^a\right)$$

Rewards are independent over
time according to the MAB
fundamental assumption

$$\rightarrow = f_{r|a}\left(r_t^a \mid Q(a)\right) f_Q\left(Q(a) \mid r_{t-1}^a, r_{t-2}^a, \dots, r_1^a\right)$$

If we have a model for the densities $f(\cdot)$, we can sample K independent values and obtain an unbiased estimate as:

$$\hat{Q}(a) = \frac{1}{K} \sum_{k=1}^K Q_k(a)$$

Thomson sampling bandit algorithm

Initialize

$t = 0$

for $i = 1:m$

$Q(i) = \text{rand}$

Loop forever

$t \leftarrow t + 1$

$A \leftarrow \arg \max_a \hat{Q}(a)$

$r_t^A \leftarrow \text{bandit}(A)$

for $i = 1:m$

for $k = 1:K$

$Q_k(i) \sim f(Q(i) | r_t^i, r_{t-1}^i, \dots, r_1^i)$

$\hat{Q}(i) \leftarrow \frac{1}{K} \sum_{k=1}^K Q_k(i)$

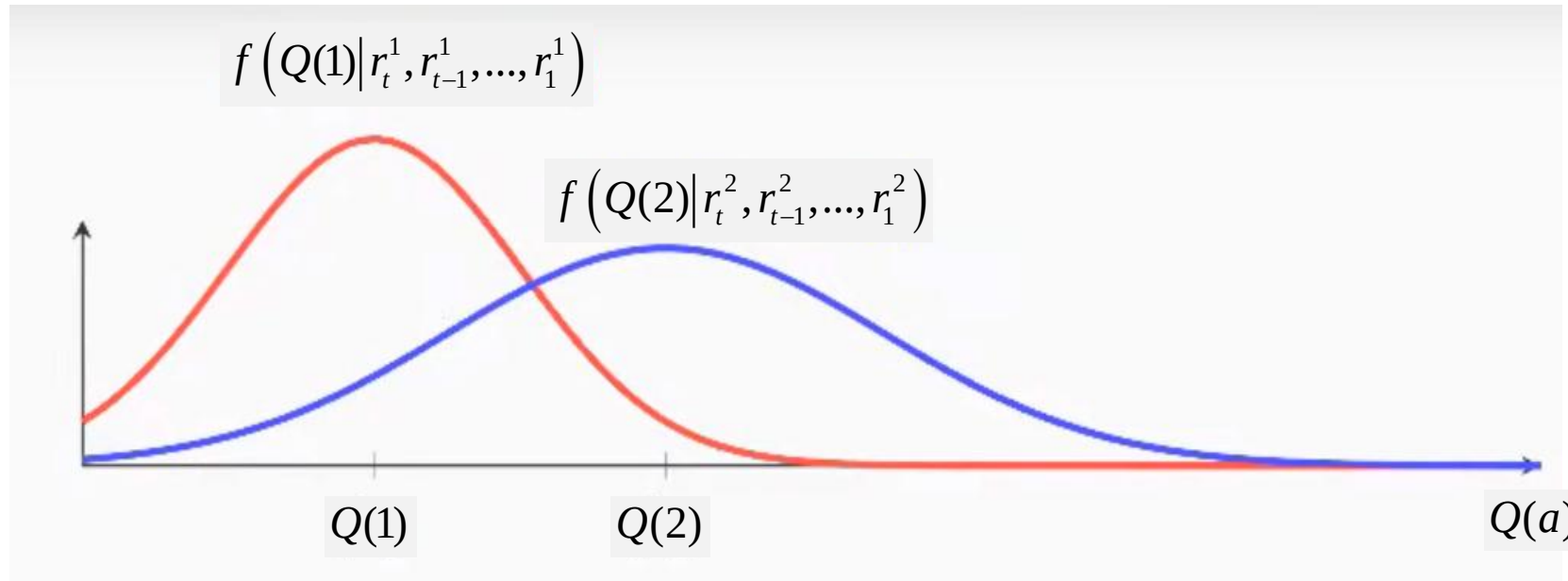
ϵ -greedy could also be used

Draw K random samples of $Q(i)$ for each action (here is why “sampling”) according to $f()$

Rewards observed for action i up to time t

Smaller K entail more uncertainty in the computation of the mean, and hence more exploration. It works better than UCB as a result of the a priori assumptions.

Why does it work?



As arm #2 has been selected less times the variance of estimated $Q(2)$ is larger, so it is likely we draw samples with more extreme value $\rightarrow$ exploration (as in UCB)

Cumulative regret is $O(\log t)$

Example 2.2. An m -armed bandit with **binary rewards**, with Beta priors

Let us see how to compute the posterior in a particular example. Assume the reward r associated to each action a has a Bernoulli density:

$$f_{r|a}(r|a) = \theta_a^r (1 - \theta_a)^{1-r} \quad a = 1, \dots, m$$

It turns out that the mean of r^a (rewards for action a) is:

$$Q(a) = E\{r^a\} = 1 \times \theta_a + 0 \times (1 - \theta_a) = \theta_a$$

Let us assume that the prior for the unknown $Q(a)$ is a Beta distribution:

$$f_Q(\theta_a | r_{t-1}^a, r_{t-2}^a, \dots, r_1^a) = \text{Beta}(\theta_a; \alpha_a, \beta_a) \propto \theta_a^{\alpha_a-1} (1 - \theta_a)^{\beta_a-1}$$

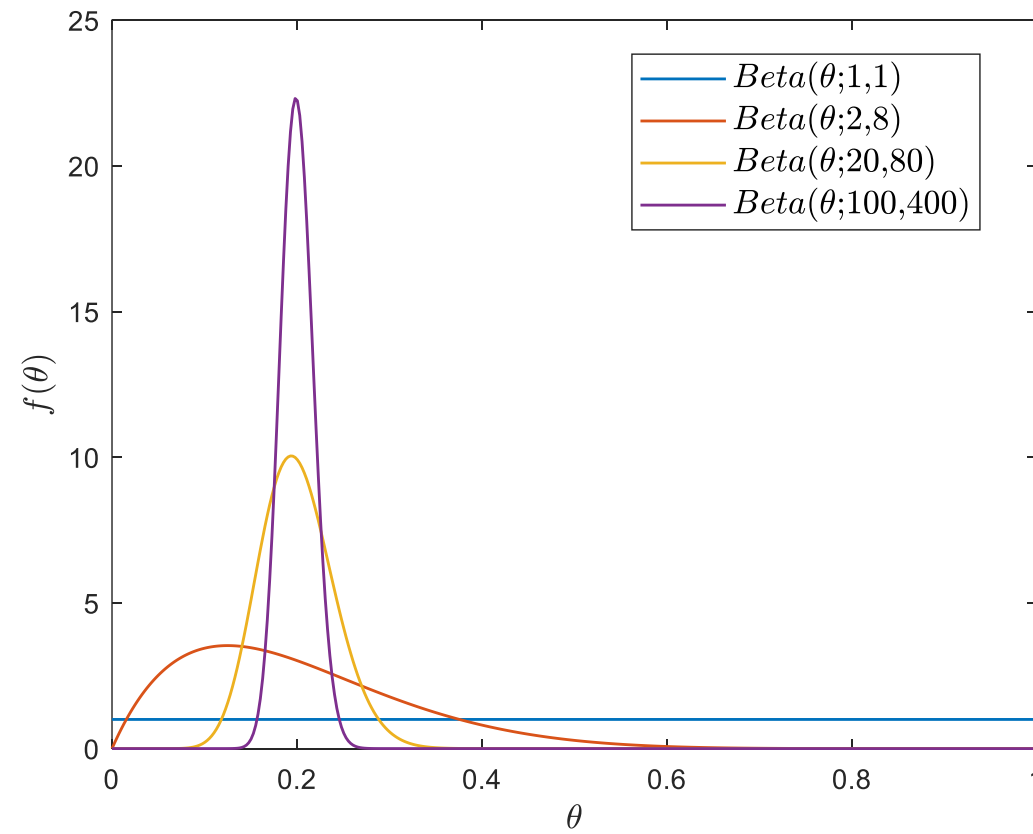
$$E\{\theta_a\} = \alpha_a / (\alpha_a + \beta_a)$$

where

$$\alpha_a - 1: \text{ \# of 1 observed in } r_{t-1}^a, r_{t-2}^a, \dots, r_1^a$$

$$\beta_a - 1: \text{ \# of 0 observed in } r_{t-1}^a, r_{t-2}^a, \dots, r_1^a$$

Note that we have different α and β for each possible action a .



Example 2.2. An m -armed bandit with **binary rewards**, with Beta priors

Let us combine the Bernoulli and the Beta densities to compute the posterior after arm a has been selected and we observed the reward:

- If reward is 1

$$\begin{aligned} f(\theta_a | r_t^a = 1, r_{t-1}^a, \dots, r_1^a) &\propto f_{r|a}(r_t^a = 1 | \theta_a) f_Q(\theta_a | r_{t-1}^a, \dots, r_1^a) \\ &\propto \theta_a \theta_a^{\alpha_a - 1} (1 - \theta_a)^{\beta_a - 1} = \text{Beta}(\theta_a; \alpha_a + 1, \beta_a) \end{aligned}$$

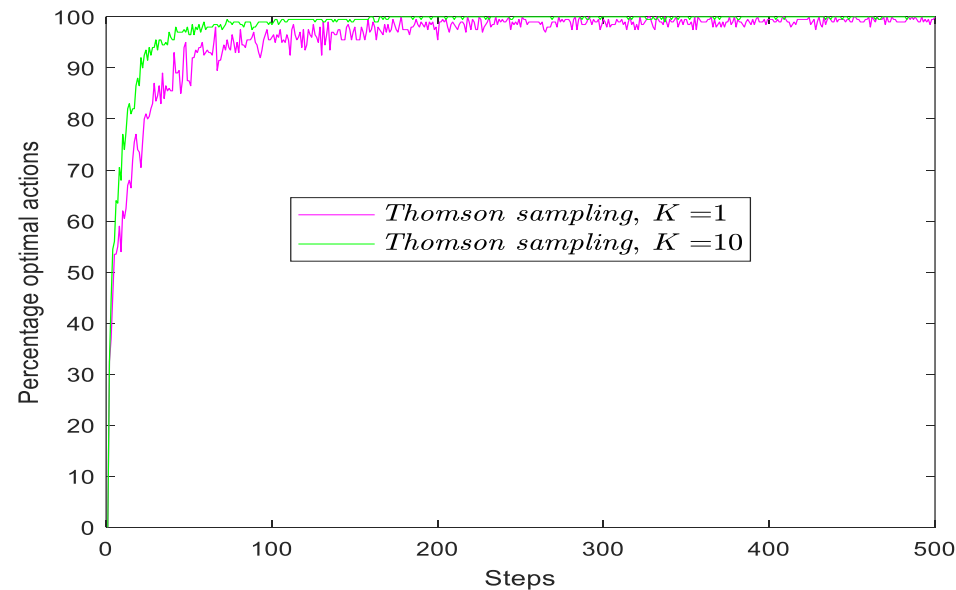
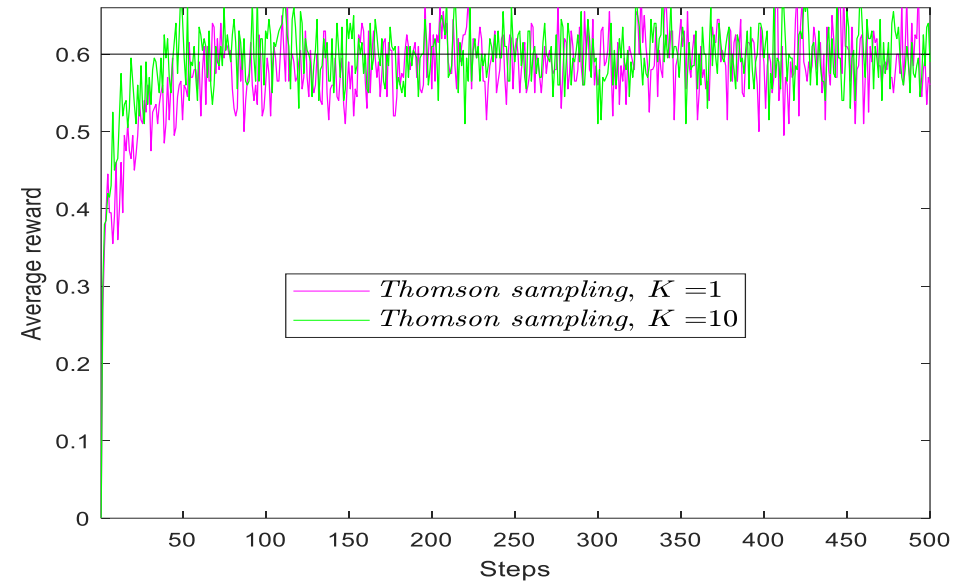
- If reward is 0

$$\begin{aligned} f(\theta_a | r_t^a = 0, r_{t-1}^a, \dots, r_1^a) &\propto f_{r|a}(r_t^a = 0 | \theta_a) f_Q(\theta_a | r_{t-1}^a, \dots, r_1^a) \\ &\propto (1 - \theta_a) \theta_a^{\alpha_a - 1} (1 - \theta_a)^{\beta_a - 1} = \text{Beta}(\theta_a; \alpha_a, \beta_a + 1) \end{aligned}$$

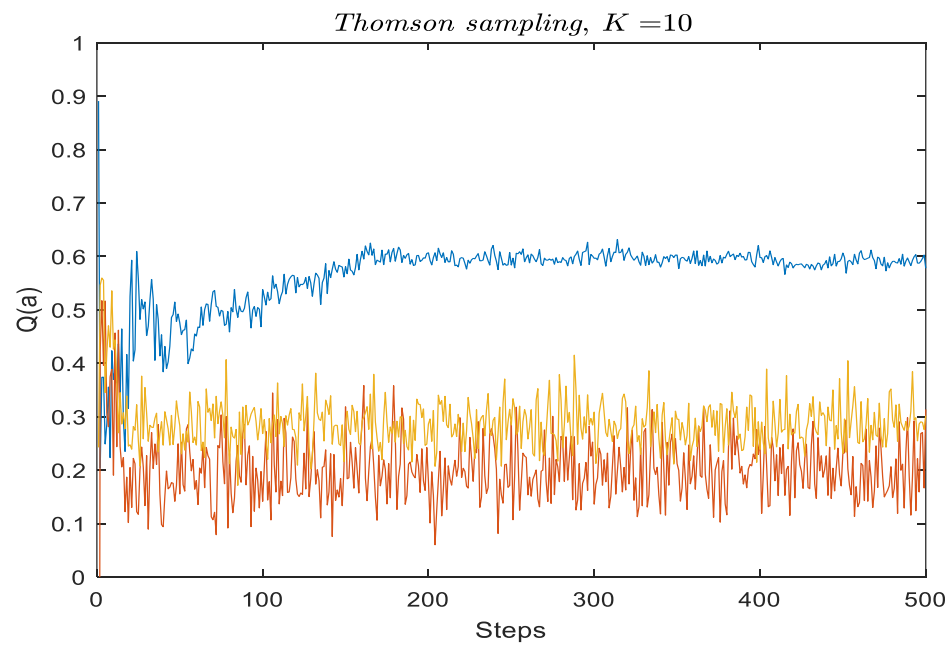
Then, we obtain Beta distributions for each arm. How to generate random samples of $Q(a)$ following a Beta distribution? In Python:

```
Q = numpy.random.beta(alpha, beta)
```

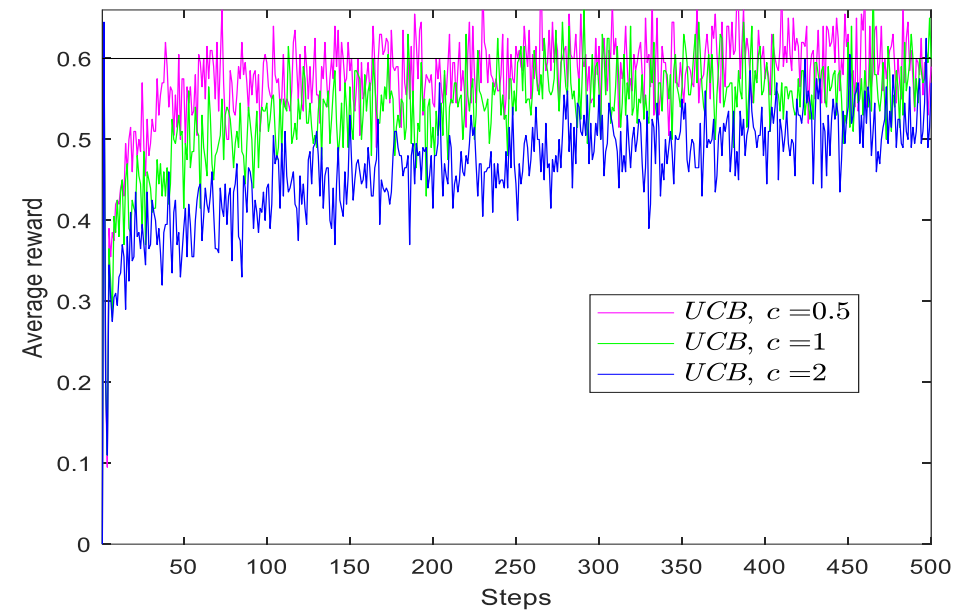
Example 2.2. Thomson sampling convergence averaged over 200 runs, for $\{0,1\}$ random reward on each of the 3 arms. $Q(a) = \theta_a = E\{r^a\}$ are $[0.6 \ 0.2 \ 0.1]$



Example 2.2. Thomson sampling convergence averaged over 200 runs, for $\{0,1\}$ random reward on each of the 3 arms. $Q(a) = \theta_a = E\{r^a\}$ are $[0.6 \ 0.2 \ 0.1]$

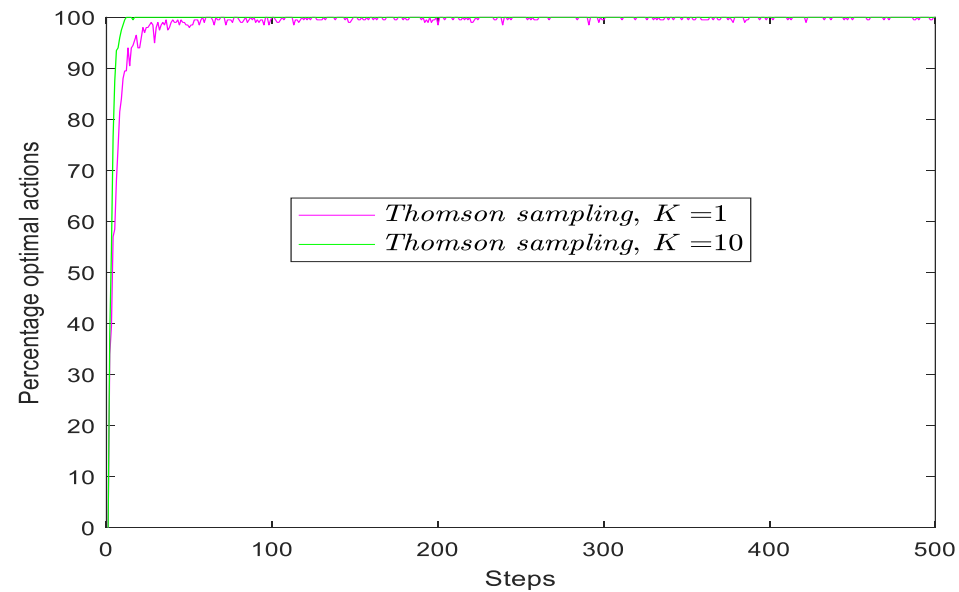
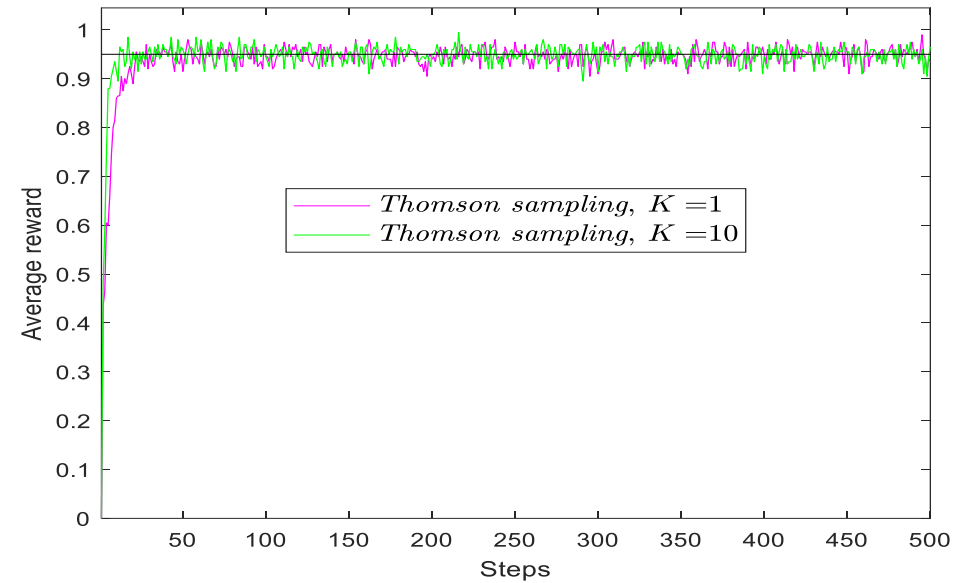


Example 2.2. UCB convergence averaged over 200 runs, for a $\{0,1\}$ random reward for each of the 3 arms. $Q(a) = \theta_a = E\{r^a\}$ are $[0.6 \ 0.2 \ 0.1]$



Thomson sampling approach exhibits faster convergence and better stability than UCB.

Example 2.2. Convergence gets better as means are more different:
 $Q(a) = \theta_a = E\{r^a\}$ are now [0.95 0.2 0.1]



Example 2.3. An m -armed bandit with **Gaussian rewards**, and Gaussian prior

Assume the reward r associated to each action a has a Gaussian density:

$$f_{r|a}(r^a|Q(a)) = \mathcal{N}(Q(a), \lambda_a^2) \quad a = 1, \dots, m$$

Let us assume that the prior for the unknown $Q(a)$ is Gaussian:

$$f_Q(Q(a)|r_{t-1}^a, r_{t-2}^a, \dots, r_1^a) = \mathcal{N}(Q(a), \sigma_a^2/(t-1)) \quad a = 1, \dots, m$$

This makes sense if taking $Q(a) = \frac{1}{t-1} (r_{t-1}^a + r_{t-2}^a + \dots + r_1^a) + w_t = \frac{1}{t-1} \mathbf{1}^T \mathbf{r}_{t-1} + w_t$ where the estimation error w_t is zero mean and its variance reduces proportionally with the number of observations. Putting everything together:

$$\begin{aligned} f(Q(a)|r_t^a, r_{t-1}^a, r_{t-2}^a, \dots, r_1^a) &\propto f_{r|a}(r_t^a|Q(a)) f_Q(Q(a)|r_{t-1}^a, r_{t-2}^a, \dots, r_1^a) \\ &\propto \exp\left(-\frac{1}{2\lambda_a^2}(r_t^a - Q(a))^2\right) \exp\left(-\frac{t-1}{2\sigma_a^2}\left(Q(a) - \frac{1}{t-1} \mathbf{1}^T \mathbf{r}_{t-1}^a\right)^2\right) \\ &\propto \exp\left(-\frac{1}{2\rho_a^2}(Q(a) - \mu_t^a)^2\right) \end{aligned}$$

Example 2.3. An m -armed bandit with **Gaussian rewards**, and Gaussian prior

By equating the last two terms and lumping whatever is independent of $Q(a)$:

$$\frac{1}{\rho_a^2} = \frac{1}{\lambda_a^2} + \frac{t-1}{\sigma_a^2}$$
$$\mu_t^a = \rho^2 \left(\frac{r_t^a}{\lambda_a^2} + \frac{\mathbf{1}^T \mathbf{r}_{t-1}^a}{\sigma_a^2} \right) = \frac{\sigma_a^2 r_t^a + \lambda_a^2 \mathbf{1}^T \mathbf{r}_{t-1}^a}{\sigma_a^2 + \lambda_a^2 (t-1)}$$

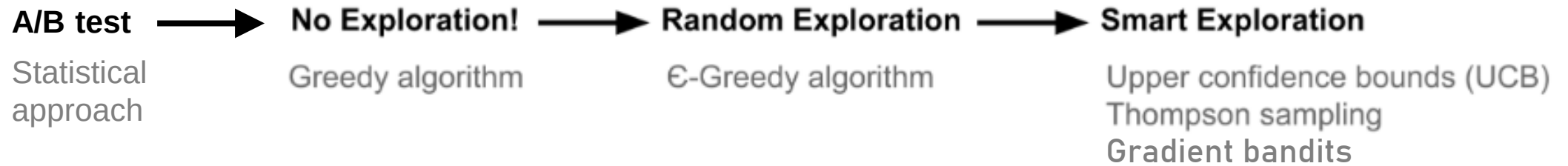
Note that t is the number of times action a has been selected, so we should write $N_t(a)$ instead. Also, we can assume $\lambda_a = \sigma_a$ and hence:

$$\frac{1}{\rho_a^2} = \frac{N_t(a)}{\sigma_a^2} \quad \mu_t^a = \frac{\mathbf{1}^T \mathbf{r}_t}{N_t(a)} = \mu_{t-1}^a + \frac{1}{N_t(a)} (r_t - \mu_{t-1}^a)$$

The variance of each action can also be computed online as:

$$\sigma_{a,t}^2 = \sigma_{a,t-1}^2 + \frac{1}{N_t(a)} \left((r_t^a - \mu_{t-1}^a)(r_t^a - \mu_t^a) - \sigma_{a,t-1}^2 \right)$$

Wrapping up...



“For an add selection in a website, in the **traditional A/B test**, I would have served my customers an inferior version of my website over 600,000 times, whereas Thompson Sampling required just over 5,000 to learn the same thing. That’s 120x fewer mistakes!”

By Greg Rafferty

Contextual bandits

Now the best possible action is tied to a context that is observed by the agent.

This is industrially exploited to generate personal user interfaces, personal advertising, personalised web contents,...



Contextual bandits

We need to learn a mapping from situations to actions, let us introduce states:

- A **contextual bandit** is defined by a tuple $\langle \mathcal{A}, \mathcal{S}, \mathcal{R} \rangle$
- $\mathcal{A}$ is a set of m actions (or “arms”)
- $\mathcal{S} = \Pr\{s\}$ is an unknown distribution over states (or “contexts”)
- At each step t
 - Environment generates state $s_t \sim \mathcal{S}$
 - Agent selects one action $a_t \in \mathcal{A}$
 - Environment generates a reward $r_t \in \mathcal{R}$
- The reward follows an unknown probability density $r \sim f(r|\mathbf{s}, a)$, without memory
- States at each step are **independent**
- The objective is to find a policy $\pi: s_t \rightarrow a$ that maximizes the cumulative reward



- In contextual bandits, the regret can be reduced as long as the context is informative of the reward distribution.
- The **action-value** function is now the mean reward for action and state:

$$Q(s, a) = E \{ r \mid S = s, A = a \}$$

- Several approaches are possible to learn $Q(s, a)$:
 - Define P cluster-head states using some kind of clustering, and solve P parallel m -armed bandits
 - Parametrize function $Q(s, a)$, e.g. linear model, neural network¹ or kernels².

¹ M. Collier, H. Urdiales Llorens, “Deep Contextual Multi-armed Bandits”, <https://arxiv.org/abs/1807.09809>

² M. Valko, N. Korda, R. Munos, “Finite-Time Analysis of Kernelised Contextual Bandits”, <https://arxiv.org/pdf/1309.6869>

Linear model for contextual bandits

- Model the action-value function with a linear approximator:

$$\hat{Q}(s, a) = \phi(s, a)^T \theta_a$$

Context information in s
(user demographics, history
of web browsing, cookies,...
and any other relevant stuff)

- We gather a number of observations and rewards:

E.g., number of
clicks in a web page

$$r_\tau^a = \phi(s_\tau, a)^T \theta_a + e_\tau \quad \tau = 1, \dots, t$$

t counts the number of times
action a has been selected

being e_τ a zero mean disturbance. We have a set of parameters per arm in vector θ_a .

Linear model for contextual bandits

- We can solve the parameters of the model by using least squares (with regularization λ , although simple pseudoinverse could also be used) over t observations:

$$\min_{\boldsymbol{\theta}_{a,t}} \left\| \mathbf{A}_{a,t} \boldsymbol{\theta}_{a,t} - \mathbf{r}_t^a \right\|_2^2 + \lambda \left\| \boldsymbol{\theta}_{a,t} \right\|_2^2$$
$$\mathbf{A}_{a,t} = \begin{bmatrix} \boldsymbol{\phi}(s_1, a)^T \\ \vdots \\ \boldsymbol{\phi}(s_t, a)^T \end{bmatrix} \quad \mathbf{r}_t^a = \begin{bmatrix} r_1^a \\ \vdots \\ r_t^a \end{bmatrix}$$

$$\hat{\boldsymbol{\theta}}_{a,t} = \left(\mathbf{A}_{a,t}^T \mathbf{A}_{a,t} + \lambda \mathbf{I} \right)^{-1} \mathbf{A}_{a,t}^T \mathbf{r}_t^a$$

This t counts the number of times a has been selected so far (was $N_t(a)$ in non-contextual MAB)

Beyond linear contextual models: Kernel CMAB

- Rewrite the LS algorithm so that the inner products $\phi(s_i, a)^T \phi(s_j, a)$ are replaced by a kernel function meeting Mercer's theorem:

$$k(\phi(s_i, a), \phi(s_j, a))$$

- The kernelized solution is (see annex 3):

$$\hat{Q}(s_t, a) = \phi(s_t, a)^T \theta_{a,t} = \phi(s_t, a)^T \mathbf{A}_{a,t}^T \mathbf{K}_{a,t}^{-1} \mathbf{r}^a = \mathbf{k}(s_t)_a^T \mathbf{K}_{a,t}^{-1} \mathbf{r}^a$$

Be careful with the implementation as $\mathbf{K}$ grows without bound with time (which is not the case with $\mathbf{A}^T \mathbf{A}$). Some sliding window mechanism has to be implemented.

Linear model for contextual bandits

Let us see how to balance between **exploitation** and **exploration** using the linear model, with the following approaches:

1. Least squares ϵ – greedy (can you define it?)
2. UCB action selection (LinUCB)
3. Probability matching (LinTS)
4. Softmax contextual bandit

Contextual ε -greedy bandit algorithm

Initialize

for $i = 1:m$

$\mathbf{A}_{i,0} = \text{empty_matrix}$

$\boldsymbol{\theta}_{i,0} = \text{random_vector}$

$t = 0$

Loop forever

$t \leftarrow t + 1$

Update ε

Observe ϕ_t

$a \leftarrow \begin{cases} \arg \max_a \phi_t^T \boldsymbol{\theta}_a & \text{with probability } 1 - \varepsilon \\ \text{random action} & \text{with probability } \varepsilon \end{cases}$

$r_t^a \leftarrow \text{bandit}(a)$

$\mathbf{A}_{a,t} = \begin{bmatrix} \mathbf{A}_{a,t-1} \\ \phi_t^T \end{bmatrix} \quad \mathbf{r}_t^a = \begin{bmatrix} \mathbf{r}_{t-1}^a \\ r_t^a \end{bmatrix}$

$\boldsymbol{\theta}_a = (\mathbf{A}_{a,t}^T \mathbf{A}_{a,t} + \lambda \mathbf{I})^{-1} \mathbf{A}_{a,t}^T \mathbf{r}_t^a$

Approach 2. UCB for contextual linear model

- Least squares approach allows computing the variance of the estimated action-value function, i.e. the uncertainty → define the **UCB algorithm for the linear model** (check Annex 4):

$$a_{T+1} \doteq \arg \max_a \left[\phi(s_{T+1}, a)^T \theta_a + c \sqrt{2 \ln T \phi(s_{T+1}, a)^T \mathbf{C}_t^{-1} \phi(s_{T+1}, a)} \right]$$

$$\mathbf{C}_t = \mathbf{A}_{a,t}^T \mathbf{A}_{a,t} + \lambda \mathbf{I}$$

$c > 0$, a higher value
entails higher level
of exploration

Confidence bound $U_t(a)$: it is related to
the uncertainty in the estimate of $Q(s, a)$

This T counts the total
time instants elapsed (was
 t in non-contextual MAB)

<https://arxiv.org/pdf/1003.0146.pdf>

- Regret bound with probability δ : $O\left(\sqrt{Td \ln^3(NT \ln(T) / \delta)}\right)$

Approach 2. UCB for contextual linear model

- Note that the LS solution can be computed recursively as time passes:

$$\mathbf{A}_{a,t} = \begin{bmatrix} \phi(s_1, a)^T \\ \vdots \\ \phi(s_t, a)^T \end{bmatrix} = \begin{bmatrix} \mathbf{A}_{a,t-1} \\ \phi(s_t, a)^T \end{bmatrix} \quad \mathbf{r}_t^a = \begin{bmatrix} r_1^a \\ \vdots \\ r_t^a \end{bmatrix} = \begin{bmatrix} \mathbf{r}_{t-1}^a \\ r_t^a \end{bmatrix}$$

$$\hat{\boldsymbol{\theta}}_{a,t} = \left(\mathbf{A}_{a,t-1}^T \mathbf{A}_{a,t-1} + \phi(s_t, a) \phi(s_t, a)^T \right)^{-1} \left(\mathbf{A}_{a,t-1}^T \mathbf{r}_{t-1}^a + \phi(s_t, a) r_t^a \right)$$

Using the recursion provided by the [Sherman-Morrison formula](#) (check Annex 5) it is possible to derive a recursive-in-time solution, and the cost of matrix inversion can be substantially reduced, especially if the size of the context vector is large.

- The Shermann-Morrison formula can be numerically unstable. For a stable solution based on QR decomposition of matrices, check also Annex 5.

Approach 3. Probability matching for contextual linear model

We will assume a Gaussian prior for the coefficients of the linear model. As for the non-contextual case, we need to generate samples of $Q(s, a)$ and average them to estimate the mean value.

We define the following terms:

Gaussian prior

$$f_{\theta}(\boldsymbol{\theta}_a) = \mathcal{N}(0; \lambda^2 \mathbf{I}) \propto \exp\left(-\frac{\boldsymbol{\theta}_a^T \boldsymbol{\theta}_a}{2\lambda^2}\right)$$

Gaussian likelihood for observed rewards

$$f_{\mathbf{r}}(\mathbf{r}_t^a | \mathbf{A}_{a,t}, \boldsymbol{\theta}_a) = \mathcal{N}(\mathbf{A}_{a,t}^T \boldsymbol{\theta}_a; \sigma^2) \propto \exp\left(-\frac{\|\mathbf{r}_t^a - \mathbf{A}_{a,t} \boldsymbol{\theta}_a\|^2}{2\sigma^2}\right)$$

Gaussian posterior

$$f(\boldsymbol{\theta}_a | \mathbf{r}_t^a, \mathbf{A}_{a,t}) \propto f(\mathbf{r}_t^a | \mathbf{A}_{a,t}, \boldsymbol{\theta}_a) f(\boldsymbol{\theta}_a) = \exp\left(-\frac{\|\mathbf{r}_t^a - \mathbf{A}_{a,t} \boldsymbol{\theta}_a\|^2}{2\sigma^2}\right) \exp\left(-\frac{\boldsymbol{\theta}_a^T \boldsymbol{\theta}_a}{2\lambda^2}\right) = \mathcal{N}(\boldsymbol{\mu}_{a,t}; \mathbf{C}_{a,t})$$

$$\boldsymbol{\mu}_{a,t} = \sigma^{-2} \mathbf{C}_{a,t} \mathbf{A}_{a,t}^T \mathbf{r}_t^a \quad \mathbf{C}_{a,t} = \left(\sigma^{-2} \mathbf{A}_{a,t} \mathbf{A}_{a,t}^T + \lambda^{-2} \mathbf{I}\right)^{-1}$$

Approach 3. Probability matching for contextual linear model

The [Sherman-Morrison formula](#) can again be used to reduce the complexity in the computation of $\boldsymbol{\mu}_a$ and $\mathbf{C}_a$.

We generate many random values of $Q(s, a)$ by using the relation:

$$Q(s, a) = \boldsymbol{\phi}(s, a)^T \boldsymbol{\theta}_a$$

the posterior for $Q(s, a)$ is given from the previous slide:

$$f\left(Q(s, a) \mid \mathbf{r}_t^a, \mathbf{A}_{a,t}\right) = f\left(\boldsymbol{\phi}(s, a)^T \boldsymbol{\theta}_a \mid \mathbf{r}_t^a, \mathbf{A}_{a,t}\right) = \mathcal{N}\left(\boldsymbol{\phi}(s, a)^T \boldsymbol{\mu}_{a,t}; \boldsymbol{\phi}(s, a)^T \mathbf{C}_{a,t} \boldsymbol{\phi}(s, a)\right)$$

Then, by averaging K sampled values, we get the estimate of the average $Q(s, a)$ we are looking for. Note that more exploration is obtained with

- Larger values of λ and σ
- Smaller values of K

A practical online service

MSN news: 26% improvement in click through rate after adopting the contextual bandits approach.

The screenshot displays the MSN news homepage with a navigation bar at the top including links like 'Mis noticias', 'Personalizar', 'Coronavirus', and various news categories. The main content area features several news tiles:

- NOTICIAS**: A large tile with a photo of a man wearing a face mask, titled 'Remiten a Podemos y Vox la declaración de Calvente' from 'El Confidencial'.
- Consejos para evitar la meningoencefalitis vírica**: A tile with a photo of a mosquito, titled 'Consejos para evitar la meningoencefalitis vírica' from 'El Mundo'.
- China detecta covid en alitas importadas desde Brasil**: A tile with a photo of fried chicken, titled 'China detecta covid en alitas importadas desde Brasil' from 'Vozpópuli'.
- Así ha cambiado Emma García**: A tile with a photo of a woman, titled 'Así ha cambiado Emma García' from 'Showbizz Daily'.
- Reconecta con tu movilidad**: A tile with a photo of two cars, titled 'Reconecta con tu movilidad'.
- Un nuevo estudio sugiere que las partículas de coronavirus suspendidas en**: A tile with a photo of a person in a hospital bed, titled 'Un nuevo estudio sugiere que las partículas de coronavirus suspendidas en'.
- Los españoles nacidos antes de 1980 se benefician de este nuevo beneficio**: A tile with a photo of a Spanish ID card, titled 'Los españoles nacidos antes de 1980 se benefician de este nuevo beneficio'.

On the right side, there is a weather widget for 'VILASSAR DE MAR, CATALUÑA' showing a temperature of 29°C and a forecast for the next few days. Below the weather widget is a sports results section for 'LALIGA SANTANDER' showing matches like SFC vs VCF (1-0) and ESP vs CEL (0-0).

<https://www.microsoft.com/en-us/research/blog/real-world-interactive-learning-cusp-enabling-new-class-applications>

Online service economy

The experimental design has to be modified as the experiment progresses to make the MAB experiment as inexpensive as possible:

<https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/42550.pdf>

The adversarial m -armed bandit problem

Let us formalize the concept...

An adversarial **multi-armed bandit** is defined by $\langle \mathcal{A}, \mathcal{R} \rangle$

- $\mathcal{A}$ is a set of $m = |\mathcal{A}|$ actions (or “arms”)
- At each step t
 - an oblivious adversary defines rewards $\mathcal{R} = \{r_{1,t}, \dots, r_{m,t}\} \in [0,1]^m$
 - agent selects one action $a_t \in \mathcal{A}$
 - agent receives a reward $r_{a_t,t} \in \mathcal{R}$
- The objective is to take appropriate decisions so as to minimize the pseudo-regret:

$$\bar{L}_t = \max_{i=1,\dots,m} E \left\{ \sum_{\tau=1}^t r_{i,\tau} - \sum_{\tau=1}^t r_{a_\tau,\tau} \right\}$$

remember that for the non-adversarial case... $L_t = E \left\{ \sum_{\tau=1}^t (V^* - r_\tau) \right\}$



Exp3 algorithm

Initialize

$t = 0$, for $i = 1:m$ $p_1(i) = 1/m$ $S_0(i) = 0$ $\eta \in (0,1]$

Loop forever

$t \leftarrow t + 1$

$A \sim p(A)$

$r \leftarrow \text{bandit}(A)$

for $i = 1:m$

$$\hat{s} = \frac{r}{p(A)} \delta_{i-A}$$

unbiased
estimate of
reward

$$S(i) \leftarrow S(i) + \hat{s}$$

$$p(i) \leftarrow (1 - \eta) \frac{\exp(\eta S(i))}{\sum_{k=1}^m \exp(\eta S(k))} + \frac{\eta}{m}$$

exploration & exploitation

Kronecker delta

If the algorithm is run with $\eta_t = \sqrt{\frac{\ln m}{tm}}$, the pseudo-regret is bounded: $\bar{L}_t \leq 2\sqrt{tm \ln m}$

<http://cseweb.ucsd.edu/~yfreund/papers/bandits.pdf>

Quiz
#1



Annex 1. Proof for UCB

A fundamental statistical result:

Hoeffding's inequality

Let $X_1, \dots, X_t$ be independent random variables bounded in $[X_i^M, X_i^m]$ and let the sample mean be:

$$\bar{X}_t = \frac{1}{t} \sum_{\tau=1}^t X_{\tau}$$

Then,

$$\Pr \left\{ \bar{X}_t - \frac{1}{t} \sum_{i=1}^t E \{ X_i \} > u \right\} \leq \exp \left(\frac{-2t^2 u^2}{\sum_{i=1}^t (X_i^M - X_i^m)^2} \right)$$

Let us apply Hoeffding's inequality to rewards of the bandit characterized by random rewards r_a on each action. If we have selected action a for $N_t(a)$ times:

$$\Pr \left\{ \hat{Q}_t(a) - Q(a) > U_t(a) \right\} \leq \exp \left(\frac{-2N_t(a)U_t^2(a)}{(r_a^M - r_a^m)^2} \right)$$

Define a confidence bound on the value of $Q(a)$ using this inequality, and solve for $U_t(a)$:

$$\exp\left(\frac{-2N_t(a)U_t^2(a)}{(r_a^M - r_a^m)^2}\right) = p$$
$$U_t(a) = \sqrt{\frac{-(r_a^M - r_a^m)^2 \ln p}{2N_t(a)}}$$

We can reduce p as we observe more rewards, e.g. $p = t^{-c}$:

$$U_t(a) = \sqrt{\frac{c(r_a^M - r_a^m)^2 \ln t}{2N_t(a)}}$$

Note that c acts as an effective way to reduce the number of visits to actions $N_t(a)$, so a larger values implies more exploration.

For a Gaussian random variable, the UCB bound can be computed from the Chernoff exponential bound on the Gaussian integral:

$$\Pr\left\{\hat{Q}_t(a) - Q(a) > U_t(a)\right\} = \int_{U_t(a)}^{\infty} \frac{1}{\sqrt{2\pi\sigma_{\hat{Q}_t(a)}^2}} \exp\left(-\frac{x^2}{2\sigma_{\hat{Q}_t(a)}^2}\right) dx \leq \exp\left(-\frac{U_t^2(a)}{\sigma_a^2 / N_t(a)}\right)$$

where we have used that the variance of the sample average estimate: $\sigma_{\hat{Q}_t(a)}^2 = \frac{\sigma_a^2}{N_t(a)}$
From here we get :

$$U_t(a) = \sqrt{\frac{c\sigma_a^2 \ln t}{N_t(a)}}$$

Annex 2. Gradient bandits

Gradient ascend principle – the preference $H(a)$ is incremented proportionally to the increment in performance:

$$H_{t+1}(a) = H_t(a) + \alpha \frac{\partial E\{r_t\}}{\partial H_t(a)}$$

where the performance measure is $E\{r_t\} = \sum_{x \in \mathcal{A}} \pi_t(x) Q(x)$

Let us compute the gradient:

$$\frac{\partial E\{r_t\}}{\partial H_t(a)} = \frac{\partial}{\partial H_t(a)} \left[\sum_{x \in \mathcal{A}} \pi_t(x) Q(x) \right] = \sum_{x \in \mathcal{A}} Q(x) \frac{\partial \pi_t(x)}{\partial H_t(a)} = \sum_{x \in \mathcal{A}} (Q(x) - B_t) \frac{\partial \pi_t(x)}{\partial H_t(a)}$$

As the gradient sums to zero over all the actions (see next slides), it is licit to introduce a baseline B_t that does not depend on x . We arbitrarily choose:

$$B_t = \bar{R}_t = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \hat{Q}(a)$$

Elaborating on the expression...

$$\frac{\partial E \{r_t\}}{\partial H_t(a)} = \sum_{x \in \mathcal{A}} \pi_t(x) (Q(x) - \bar{R}_t) \frac{\partial \pi_t(x)}{\partial H_t(a)} \frac{1}{\pi_t(x)}$$

$$= E \left\{ \left(Q(A_t) - \bar{R}_t \right) \frac{\partial \pi_t(A_t)}{\partial H_t(a)} \frac{1}{\pi_t(A_t)} \right\}$$

Indicator function:
1 if the argument is true,
0 otherwise

$$= E \left\{ \left(r_t - \bar{R}_t \right) \frac{\partial \pi_t(A_t)}{\partial H_t(a)} \frac{1}{\pi_t(A_t)} \right\}$$

since $E \{r_t | A_t\} = Q(A_t)$

$$= E \left\{ \left(r_t - \bar{R}_t \right) \pi_t(A_t) \left(1_{a=x} - \pi_t(a) \right) \frac{1}{\pi_t(A_t)} \right\}$$

proved in next slide

$$= E \left\{ \left(r_t - \bar{R}_t \right) \left(1_{a=A_t} - \pi_t(a) \right) \right\}$$

Now substitute the expectation by the argument to obtain the stochastic-like algorithm proposed:

$$H_{t+1}(A_t) = H_t(A_t) + \alpha (r_t - \bar{R}_t) (1_{a=A_t} - \pi_t(a)) \quad \text{for all } a$$

Let us finally prove the expression for the gradient:

$$\begin{aligned}
\frac{\partial \pi_t(x)}{\partial H_t(a)} &= \frac{\partial}{\partial H_t(a)} \left[\frac{\exp(H_t(x))}{\sum_{y \in \mathcal{A}} \exp(H_t(y))} \right] \\
&= \frac{\frac{\partial \exp(H_t(x))}{\partial H_t(a)} \sum_{y \in \mathcal{A}} \exp(H_t(y)) - \exp(H_t(x)) \frac{\partial \sum_{y \in \mathcal{A}} \exp(H_t(y))}{\partial H_t(a)}}{\left(\sum_{y \in \mathcal{A}} \exp(H_t(y)) \right)^2} \\
&= \frac{1_{a=x} \exp(H_t(x)) \sum_{y \in \mathcal{A}} \exp(H_t(y)) - \exp(H_t(x)) \exp(H_t(a))}{\left(\sum_{y \in \mathcal{A}} \exp(H_t(y)) \right)^2} \\
&= \frac{1_{a=x} \exp(H_t(x))}{\sum_{y \in \mathcal{A}} \exp(H_t(y))} - \frac{\exp(H_t(x)) \exp(H_t(a))}{\left(\sum_{y \in \mathcal{A}} \exp(H_t(y)) \right)^2} \\
&= 1_{a=x} \pi_t(x) - \pi_t(x) \pi_t(a) \\
&= \pi_t(x) (1_{a=x} - \pi_t(a))
\end{aligned}$$

Annex 3. Kernel CMAB

Since the MSE is a convex function, we can write the LS solution as the convergent point of the gradient descent algorithm (simplifying the notation to avoid fuzziness):

$$\boldsymbol{\theta}_n = \boldsymbol{\theta}_{n-1} - \delta \nabla_{\boldsymbol{\theta}} \text{MSE} = \boldsymbol{\theta}_{n-1} - 2\delta \left(\mathbf{A}^T \mathbf{A} \hat{\boldsymbol{\theta}}_{n-1} - \mathbf{A}^T \mathbf{r} \right) = \boldsymbol{\theta}_{n-1} - \delta \mathbf{A}^T \mathbf{y}_{n-1}$$

Therefore, starting at $\boldsymbol{\theta}_0 = \mathbf{0}$, at all iterations $\boldsymbol{\theta}_n$ will be a linear combination of columns of $\mathbf{A}^T$. As a consequence, the LS solution can be written:

$$\mathbf{A}^T \boldsymbol{\alpha} = \boldsymbol{\theta}_{LS} = \left(\mathbf{A}^T \mathbf{A} \right)^{-1} \mathbf{A}^T \mathbf{r}$$

$$\boldsymbol{\theta}_{LS} = \mathbf{A}^T \boldsymbol{\alpha} \quad \Rightarrow$$

$$\mathbf{A} \mathbf{A}^T \mathbf{A} \mathbf{A}^T \boldsymbol{\alpha} = \mathbf{A} \mathbf{A}^T \mathbf{A} \left(\mathbf{A}^T \mathbf{A} \right)^{-1} \mathbf{A}^T \mathbf{r}$$

Kernelizing means changing
the rows of $\mathbf{A}$ by infinite-size
vectors $\boldsymbol{\phi}$, and make $\mathbf{K}$ full rank

$$\mathbf{K}^2 \boldsymbol{\alpha}_k = \mathbf{K} \mathbf{r} \quad \Rightarrow \quad \boldsymbol{\alpha}_k = \mathbf{K}^{-1} \mathbf{r}$$

$$[\mathbf{K}]_{ij} = \boldsymbol{\phi}_i^T \boldsymbol{\phi}_j = k(\boldsymbol{\phi}_i, \boldsymbol{\phi}_j)$$

When evaluating a new context vector using the kernelized LS:

$$\hat{Q}(s) = \boldsymbol{\phi}(s)^T \boldsymbol{\theta}_{kLS} = \boldsymbol{\phi}(s)^T \mathbf{A}^T \boldsymbol{\alpha}_k = \boldsymbol{\phi}(s)^T \mathbf{A}^T \mathbf{K}^{-1} \mathbf{r} = \mathbf{k}(s)^T \mathbf{K}^{-1} \mathbf{r}$$

where the components of $\mathbf{k}$ are the kernels between the current $\boldsymbol{\phi}(s)$ and the past $\boldsymbol{\phi}(s_i)$ in $\mathbf{A}$:

$$[\mathbf{k}(s)]_i = k(\boldsymbol{\phi}(s), \boldsymbol{\phi}(s_i))$$

Annex 4. UCB for linear contextual bandits

Again, we want to compute the bound on the estimation of $Q(s,a)$:

$$\Pr\left\{\hat{Q}_{t+1}(s,a) - Q(s,a) > U_{t+1}(a)\right\} \leq p$$

assuming now that the mean reward fits a linear model: $Q(s,a) = \phi(s,a)^T \theta_a$
while the observed rewards are $r_\tau^a = \phi(s_\tau, a)^T \theta_a + e_\tau$

Using the t observed rewards for action a we can determine θ using LS:

$$\theta_{a,t} = \left(\mathbf{A}_{a,t}^T \mathbf{A}_{a,t}\right)^{-1} \mathbf{A}_{a,t}^T \mathbf{r}_t^a$$

Therefore, the estimated mean reward at time $t+1$ is given by:

$$\hat{Q}_{t+1}(s,a) = \phi(s_{t+1}, a)^T \theta_{a,t}$$

Let us now statistically characterize $\hat{Q}_{t+1}(s,a)$ in terms of bias and variance.

The estimate is unbiased according to our model:

$$\begin{aligned} E \left\{ \hat{Q}_{t+1}(s, a) \right\} &= E \left\{ \phi(s_{t+1}, a)^T \boldsymbol{\theta}_{a,t} \right\} = \phi(s_{t+1}, a)^T E \left\{ \boldsymbol{\theta}_{a,t} \right\} = \phi(s_{t+1}, a)^T \left(\mathbf{A}_{a,t}^T \mathbf{A}_{a,t} \right)^{-1} \mathbf{A}_{a,t}^T E \left\{ \mathbf{r}_t \right\} \\ &= \phi(s_{t+1}, a)^T \left(\mathbf{A}_{a,t}^T \mathbf{A}_{a,t} \right)^{-1} \mathbf{A}_{a,t}^T \mathbf{A}_{a,t} \boldsymbol{\theta}_a = \phi(s_{t+1}, a)^T \boldsymbol{\theta}_a \end{aligned}$$

The variance depends on the covariance matrix of the LS estimate:

$$\begin{aligned} \text{var} \left\{ \hat{Q}_{t+1}(s, a) \right\} &= E \left\{ \left| \hat{Q}_{t+1}(s, a) - \phi(s_{t+1}, a)^T \boldsymbol{\theta}_a \right|^2 \right\} = E \left\{ \left| \phi(s_{t+1}, a)^T \boldsymbol{\theta}_{a,t} - \phi(s_{t+1}, a)^T \boldsymbol{\theta}_a \right|^2 \right\} \\ &= \phi(s_{t+1}, a)^T E \left\{ \left| \boldsymbol{\theta}_{a,t} - \boldsymbol{\theta}_a \right|^2 \right\} \phi(s_{t+1}, a) = \phi(s_{t+1}, a)^T \text{cov} \left\{ \boldsymbol{\theta}_{a,t} \right\} \phi(s_{t+1}, a) \\ \text{cov} \left\{ \boldsymbol{\theta}_{a,t} \right\} &= \left(\mathbf{A}_{a,t}^T \mathbf{A}_{a,t} \right)^{-1} \mathbf{A}_{a,t}^T E \left\{ \mathbf{w}_{a,t} \mathbf{w}_{a,t}^T \right\} \mathbf{A}_{a,t} \left(\mathbf{A}_{a,t}^T \mathbf{A}_{a,t} \right)^{-1} = \sigma_{w_a}^2 \left(\mathbf{A}_{a,t}^T \mathbf{A}_{a,t} \right)^{-1} \end{aligned}$$

if rewards are independent among observations (basic assumption in MAB).

Now let us assume that rewards are Gaussian, and so is the estimated mean reward:

$$\hat{Q}_{t+1}(s, a) \sim \mathcal{N} \left(\phi(s_{t+1}, a)^T \boldsymbol{\theta}_a; \sigma_{w_a}^2 \phi(s_{t+1}, a)^T \left(\mathbf{A}_{a,t}^T \mathbf{A}_{a,t} \right)^{-1} \phi(s_{t+1}, a) \right)$$

We can now rewrite:

$$\Pr\left\{\hat{Q}_{t+1}(s, a) - Q(s, a) > U_{t+1}(a)\right\} = \int_{U_{a,t+1}}^{\infty} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{x^2}{2\sigma^2}\right) dx$$

where $\sigma^2 = \sigma_{w_a}^2 \boldsymbol{\phi}(s_{t+1}, a)^T \left(\mathbf{A}_{a,t}^T \mathbf{A}_{a,t}\right)^{-1} \boldsymbol{\phi}(s_{t+1}, a)$.

Changing variables we can use the Gaussian integral function $\mathcal{Q}$ and upper bound it by the exponential (Chernoff bound):

$$\begin{aligned} \Pr\left\{\hat{Q}_{t+1}(s, a) - Q(s, a) > U_{t+1}(a)\right\} &= \mathcal{Q}\left(U_{t+1}(a) \left(\sigma_{w_a}^2 \boldsymbol{\phi}(s_{t+1}, a)^T \left(\mathbf{A}_{a,t}^T \mathbf{A}_{a,t}\right)^{-1} \boldsymbol{\phi}(s_{t+1}, a)\right)^{-1/2}\right) \\ &\leq \exp\left(-U_{t+1}^2(a) \left(\sigma_{w_a}^2 \boldsymbol{\phi}(s_{t+1}, a)^T \left(\mathbf{A}_{a,t}^T \mathbf{A}_{a,t}\right)^{-1} \boldsymbol{\phi}(s_{t+1}, a)\right)^{-1}\right) \leq p \end{aligned}$$

$$\text{From here: } U_{t+1}(a) = \left(-\ln p \sigma_{w_a}^2 \boldsymbol{\phi}(s_{t+1}, a)^T \left(\mathbf{A}_{a,t}^T \mathbf{A}_{a,t}\right)^{-1} \boldsymbol{\phi}(s_{t+1}, a)\right)^{1/2}$$

We can reduce the value of p as we get more rewards, to signal that we are more confident on the value of $Q(s, a)$ as time passes, e.g. $p = t^{-c}$.

Annex 5. Pseudoinverse of matrix $\mathbf{A}_{a,t}$

We need to compute the pseudoinverse of $\mathbf{A}_{a,t}$, which contains the states and grows over time:

$$\boldsymbol{\theta}_{a,t} = \left(\mathbf{A}_{a,t}^T \mathbf{A}_{a,t} \right)^{-1} \mathbf{A}_{a,t}^T \mathbf{r}_t^a \quad \mathbf{A}_{a,t} = \begin{bmatrix} \mathbf{A}_{a,t-1} \\ \boldsymbol{\phi}(s_t, a)^T \end{bmatrix}$$

Using the recursion provided by the [Sherman-Morrison formula](#) in this way:

$$\begin{aligned} \mathbf{B}_{a,t} &= \left(\begin{bmatrix} \mathbf{A}_{a,t-1}^T & \boldsymbol{\phi}(s_t, a) \end{bmatrix} \begin{bmatrix} \mathbf{A}_{a,t-1} \\ \boldsymbol{\phi}(s_t, a)^T \end{bmatrix} \right)^{-1} = \left(\mathbf{A}_{a,t-1}^T \mathbf{A}_{a,t-1} + \boldsymbol{\phi}(s_t, a) \boldsymbol{\phi}(s_t, a)^T \right)^{-1} = \\ &= \left(\mathbf{A}_{a,t-1}^T \mathbf{A}_{a,t-1} \right)^{-1} - \frac{\left(\mathbf{A}_{a,t-1}^T \mathbf{A}_{a,t-1} \right)^{-1} \boldsymbol{\phi}(s_t, a) \boldsymbol{\phi}(s_t, a)^T \left(\mathbf{A}_{a,t-1}^T \mathbf{A}_{a,t-1} \right)^{-1}}{1 + \boldsymbol{\phi}(s_t, a)^T \left(\mathbf{A}_{a,t-1}^T \mathbf{A}_{a,t-1} \right)^{-1} \boldsymbol{\phi}(s_t, a)} = \\ &= \mathbf{B}_{a,t-1} - \frac{\mathbf{B}_{a,t-1} \boldsymbol{\phi}(s_t, a) \boldsymbol{\phi}(s_t, a)^T \mathbf{B}_{a,t-1}}{1 + \boldsymbol{\phi}(s_t, a)^T \mathbf{B}_{a,t-1} \boldsymbol{\phi}(s_t, a)} \end{aligned}$$

$$\text{and } \mathbf{z}_{a,t} = \mathbf{A}_{a,t}^T \mathbf{r}_t = \begin{bmatrix} \mathbf{A}_{a,t-1}^T & \boldsymbol{\phi}(s_t, a) \end{bmatrix} \begin{bmatrix} \mathbf{r}_{t-1}^a \\ r_t \end{bmatrix} = \mathbf{z}_{a,t-1} + \boldsymbol{\phi}(s_t, a) r_t$$

Diagonal loading $\left(\mu \mathbf{I} + \mathbf{A}_{a,t}^T \mathbf{A}_{a,t}\right)^{-1}$ is needed to guarantee the existence of the inverse: note that for small t , the number of rows of $\mathbf{A}_{a,t}$ is smaller than the number of columns. This means that initially we need:

$$\mathbf{B}_{a,1} = \left(\mu \mathbf{I} + \phi(s_1, a) \phi(s_1, a)^T\right)^{-1}$$

with a small value of μ .

Recursive pseudoinverse of matrix $\mathbf{A}_{a,t}$ with limited memory

We are now interested in finding the vector $\boldsymbol{\theta}$ that minimizes this error $\mathbf{e}_{a,t}^T \mathbf{W}_t^T \mathbf{W}_t \mathbf{e}_{a,t}$ where

$$\mathbf{W}_t = \text{diag}(\lambda^t \quad \lambda^{t-1} \quad \dots \quad 1) = \begin{bmatrix} \lambda \mathbf{W}_{t-1} & \mathbf{0} \\ \mathbf{0} & 1 \end{bmatrix} \quad \text{and} \quad 0 < \lambda \leq 1$$

The solution is: $\boldsymbol{\theta}_{a,t} = (\mathbf{A}_{a,t}^T \mathbf{W}_t^T \mathbf{W}_t \mathbf{A}_{a,t})^{-1} \mathbf{A}_{a,t}^T \mathbf{W}_t^T \mathbf{W}_t \mathbf{r}_t^a$ with $\mathbf{A}_{a,t} = \begin{bmatrix} \mathbf{A}_{a,t-1} \\ \boldsymbol{\phi}(s_t, a)^T \end{bmatrix}$

Using the recursion provided by the [Sherman-Morrison formula](#):

$$\begin{aligned} \mathbf{B}_{a,t} &= \left(\begin{bmatrix} \mathbf{A}_{a,t-1}^T & \boldsymbol{\phi}(s_t, a) \end{bmatrix} \mathbf{W}_t^T \mathbf{W}_t \begin{bmatrix} \mathbf{A}_{a,t-1} \\ \boldsymbol{\phi}(s_t, a)^T \end{bmatrix} \right)^{-1} = \left(\lambda^2 \mathbf{A}_{a,t-1}^T \mathbf{W}_{t-1}^T \mathbf{W}_{t-1} \mathbf{A}_{a,t-1} + \boldsymbol{\phi}(s_t, a) \boldsymbol{\phi}(s_t, a)^T \right)^{-1} = \\ &= \left(\lambda^2 \mathbf{A}_{a,t-1}^T \mathbf{W}_{t-1}^T \mathbf{W}_{t-1} \mathbf{A}_{a,t-1} \right)^{-1} - \frac{\left(\lambda^2 \mathbf{A}_{a,t-1}^T \mathbf{W}_{t-1}^T \mathbf{W}_{t-1} \mathbf{A}_{a,t-1} \right)^{-1} \boldsymbol{\phi}(s_t, a) \boldsymbol{\phi}(s_t, a)^T \left(\lambda^2 \mathbf{A}_{a,t-1}^T \mathbf{W}_{t-1}^T \mathbf{W}_{t-1} \mathbf{A}_{a,t-1} \right)^{-1}}{1 + \boldsymbol{\phi}(s_t, a)^T \left(\lambda^2 \mathbf{A}_{a,t-1}^T \mathbf{W}_{t-1}^T \mathbf{W}_{t-1} \mathbf{A}_{a,t-1} \right)^{-1} \boldsymbol{\phi}(s_t, a)} = \\ &= \frac{1}{\lambda^2} \left(\mathbf{B}_{a,t-1} - \frac{\mathbf{B}_{a,t-1} \boldsymbol{\phi}(s_t, a) \boldsymbol{\phi}(s_t, a)^T \mathbf{B}_{a,t-1}}{\lambda^2 + \boldsymbol{\phi}(s_t, a)^T \mathbf{B}_{a,t-1} \boldsymbol{\phi}(s_t, a)} \right) \end{aligned}$$

$$\mathbf{z}_{a,t} = \mathbf{A}_{a,t}^T \mathbf{W}_t^T \mathbf{W}_t \mathbf{r}_t = \begin{bmatrix} \mathbf{A}_{a,t-1}^T & \boldsymbol{\phi}(s_t, a) \end{bmatrix} \mathbf{W}_t^T \mathbf{W}_t \begin{bmatrix} \mathbf{r}_{t-1}^a \\ r_t \end{bmatrix} = \lambda^2 \mathbf{z}_{a,t-1} + \boldsymbol{\phi}(s_t, a) r_t$$

Square root recursive pseudoinverse of matrix $\mathbf{A}_{a,t}$

The Sherman-Morrison formula is numerically unstable: it may yield a matrix $\mathbf{B}$ that is not positive definite due to finite numbers representation. It is more convenient to use an algorithm based on the QR decomposition of $\mathbf{A}$. Removing the subscript a for simplicity:

$$\min_{\boldsymbol{\theta}} \|\mathbf{r}_t - \mathbf{A}_t \boldsymbol{\theta}\|^2 = \min_{\boldsymbol{\theta}} \|\mathbf{Q}^T (\mathbf{r}_t - \mathbf{A}_t \boldsymbol{\theta})\|^2$$

where $\mathbf{Q}$ is an orthogonal matrix such that $\mathbf{A}_t = \mathbf{Q}_t \mathbf{R}_t$ with $L \leq t$, where $\mathbf{R}$ is upper-triangular.

$t \times L \quad t \times L \quad L \times L$

The QR decomposition always exist. Under this formulation:

$$\boldsymbol{\theta}_{LS,t} = (\mathbf{A}_t^T \mathbf{A}_t)^{-1} \mathbf{A}_t^T \mathbf{r}_t = \mathbf{R}_t^{-1} \mathbf{Q}_t^T \mathbf{r}_t = \mathbf{R}_t^{-1} \mathbf{d}_t$$

where $\boldsymbol{\theta}$ can be obtained very efficiently using backsubstitution because $\mathbf{R}_t$ is upper triangular. Assume we have initial $\mathbf{Q}_t$, $\mathbf{R}_t$ and $\mathbf{d}_t$. These can be updated as follows:

$$\left[\begin{array}{c|c} \mathbf{A}_{t+1} & \mathbf{r}_{t+1} \end{array} \right] = \left[\begin{array}{c|c|c} \mathbf{Q}_t & \mathbf{q}_t & \mathbf{0} \\ \hline \mathbf{0}^T & & 1 \end{array} \right] \left[\begin{array}{c|c} \mathbf{R}_t & \mathbf{d}_t \\ \hline \mathbf{0}^T & \xi_t \\ \hline \phi(s_{t+1})^T & r_{t+1} \end{array} \right] = \left[\begin{array}{c|c|c} \mathbf{Q}_t & \mathbf{q}_t & \mathbf{0} \\ \hline \mathbf{0}^T & & 1 \end{array} \right] \underbrace{\tilde{\mathbf{Q}}_{t+1} \left[\begin{array}{c|c} \mathbf{R}_{t+1} & \mathbf{d}_{t+1} \\ \hline \mathbf{0}^T & \xi_{t+1} \\ \hline \mathbf{0}^T & * \end{array} \right]}_{\text{QR decomposition of the second matrix}}$$

The product of the two unitary matrices is also a unitary matrix, so overall we have solved the QR decomposition for time $t + 1$. We do not need to explicitly compute the product of orthogonal matrices, nor the terms ξ or $*$: given initial $\mathbf{R}_t$ and $\mathbf{d}_t$ we just need to triangularise the matrix where the newly observed state vector and reward for $t + 1$ have been appended:

$$\left[\begin{array}{c|c} \mathbf{R}_t & \mathbf{d}_t \\ \hline \phi(s_{t+1})^T & r_{t+1} \end{array} \right] = \tilde{\mathbf{Q}}_{t+1} \left[\begin{array}{c|c} \mathbf{R}_{t+1} & \mathbf{d}_{t+1} \\ \hline \mathbf{0}^T & * \end{array} \right]$$

Note that the middle row that contains ξ has been just neglected. The computation is very simple since we just need to null L terms in the last row by doing linear combinations of the upper rows. This way we readily obtain $\mathbf{R}_{t+1}$ and $\mathbf{d}_{t+1}$. If we use exponential weighting for non-stationary situations, then the appended matrix is:

$$\left[\begin{array}{c|c} \lambda \mathbf{R}_t & \lambda \mathbf{d}_t \\ \hline \phi(s_{t+1})^T & r_{t+1} \end{array} \right] = \tilde{\mathbf{Q}}_{t+1} \left[\begin{array}{c|c} \mathbf{R}_{t+1} & \mathbf{d}_{t+1} \\ \hline \mathbf{0}^T & * \end{array} \right]$$

References for further study...

- Practical applications for multi-armed bandits: <https://arxiv.org/pdf/1904.10040.pdf>
- MAB in Google Analytics: <https://analytics.googleblog.com/2013/01/multi-armed-bandit-experiments.html>
- <https://banditalgs.com/>
- <https://towardsdatascience.com/a-b-testing-is-there-a-better-way-an-exploration-of-multi-armed-bandits-98ca927b357d>
- Proof of UCB bounds: <https://courses.cs.washington.edu/courses/cse599i/18wi/resources/lecture3/lecture3.pdf>
- Exploration and exploitation (by David Silver):
https://www.youtube.com/watch?v=sGuiWX07sKw&list=PLqYmG7hTraZBiG_XpjnPrSNw-1XQaM_gB&index=9
- Contextual bandits:
<https://towardsdatascience.com/contextual-bandits-and-reinforcement-learning-6bdfeaece72a>
<https://www.youtube.com/watch?v=jlcbEZTgisQ>
[Contextual Bandit: from Theory to Applications. - Vernade - Workshop 3 - CEB T1 2019 - YouTube](#)
- Bayesian multi-armed bandits (by Nando de Freitas): <https://www.youtube.com/watch?v=vz3D36VXefI>
- Regret analysis of stochastic and non-stochastic multi-armed bandits <https://arxiv.org/pdf/1204.5721v2.pdf>
- Adversarial bandits: <https://jeremykun.com/2013/11/08/adversarial-bandits-and-the-exp3-algorithm/>